

A Cognitive Substrate for Coding Agents

Theory → Evidence → Build-Map edition. Why a frozen language model structurally lacks memory, learning, imagination, self-correction, and impact-awareness — plus six metacognitive deficits the field's evidence names — and how to re-wrap its input→process→output loop to supply them, with two runnable prototypes as evidence.

A first-principles design study, grounded in field evidence. The five faculties are argued from the shape of a frozen model and grounded in 32 primary sources (memory-augmentation, continual-learning, self-correction, world-model, and program-analysis literatures). This edition adds an **evidence layer** — twelve load-bearing pain-point statistics independently re-grounded from primary sources and graded *confirmed* / *vendor-reported* / *unverifiable* — **six mechanisms** (routing, assumption gate, decomposition, goal-anchoring, anti-over-engineering, inline verification), a **build-opportunity map** against the real 2026 Claude-Code stack, and a **second prototype** (a complexity-aware router + assumption gate, evaluated live on real models). An epistemological and ethical lens drawn from Qur'anic sources runs throughout, used as design framing, never as technical authority. | Companion artifacts: evidence map, ecosystem map, gap-map, reference list, Qur'anic-lens table, and two runnable packages (`impact-oracle` , `router-gate`).

ABSTRACT

A large language model at inference time is, mathematically, a fixed function $y = f_{\theta}(x)$ with frozen parameters θ and a bounded input window. From this single fact, five apparent “cognitive” deficits of a coding agent follow as *structural consequences*, not incidental weaknesses: it cannot **remember** across sessions, cannot **learn** from outcomes, cannot **imagine** the consequences of an action before taking it, cannot reliably **correct itself**, and does not **know what already exists in a codebase or what an edit will affect**. We show that neither better prompting nor additional tools (skills, MCP servers) remove these deficits, because they leave f_{θ} and the open-loop pipeline intact. We then specify a **cognitive substrate**: an external architecture that keeps the LLM frozen but re-wraps its input→process→output loop into a closed, stateful cycle over persistent stores — an episodic/semantic memory, an online-updatable learning layer, a consequence simulator, a metacognitive verification gate, and a persistent structural model of the codebase — all under an explicit stewardship boundary. For each faculty we identify precisely what the existing literature solves and what residual gap remains for a coding agent. To turn the

weakest-evidenced claim into something testable, we build and evaluate the **impact-awareness** faculty as a runnable prototype: a Codebase World-Model that parses a repository into a persistent dependency graph, and an Impact Oracle that predicts the *blast radius* of a proposed edit. Against mutation-derived ground truth, the oracle is the only method that never misses an affected file (recall = 1.00 across five tested edits), where a text-search baseline misses transitive dependents and an edited-file-only baseline misses 47% of impact. Throughout, a Qur'anic epistemic lens supplies the design's vocabulary of obligation — know what exists before acting (2:31–32), verify before you act (49:6), pursue not that of which you have no knowledge (17:36), and hold what you can damage as a trust (33:72).

- | | |
|---|--|
| 1 The problem, from the ground | 8 Prototype I: world-model & impact oracle |
| 2 The root cause, formally | |
| 3 Five faculties and their gaps | 9 Prototype II: router & assumption gate |
| 4 The evidence: what the field measures | 10 The build-opportunity map |
| 5 The Qur'anic epistemic lens | 11 Genuinely new vs. reinvented |
| 6 Six mechanisms the frozen loop lacks | 12 Limitations & threats to validity |
| 7 The cognitive substrate | 13 Conclusion |
| | — References, evidence & lens appendices |

1 The problem, from the ground

Start with a scene any developer knows. You open an unfamiliar file. Before you change a line, a great deal happens that you barely notice: you recognize what the file *is* and how it connects to the rest of the system; you recall that you touched something similar last week and how that went; you silently simulate — “if I rename this, the three callers over there break” — and you feel the weight of the fact that this code runs in production. You are, in one glance, exercising **memory**, a **world-model**, **imagination**, and a sense of **consequence**. Only then do you type.

A coding agent built on a language model does none of this by default, and the reason is not that the model is small or under-trained. It is that the model has the wrong *shape* for the job. This paper is an attempt to go to the root of that shape — to ask what the input to the model actually is, what happens to it, what comes out, and why that pipeline cannot, on its own, hold the faculties above — and then to design a supporting structure that can, without pretending to rebuild the model or to mimic a human brain. We do not claim to make a machine that thinks like a person. We claim something narrower and buildable: that the specific faculties a coding agent is missing can be given precise mathematical form, and supplied by an external architecture wrapped around a frozen model.

Two commitments run through the paper. First, **honesty about what is known**. Some of these faculties — notably persistent memory with a real forgetting policy, and online learning without catastrophic forgetting — are open research problems; we say so, and we say exactly where the open edge is. Others — notably impact-awareness — turn out to be mostly an *engineering* gap: the tools have existed since the 1980s but were never wired into a language model's loop. We prototype that one, because it is the one where a single session can produce real evidence rather than a promise. Second, a **lens**. This project was asked to think with the Qur'an as a source of epistemology and ethics — not as authority for any engineering claim, but as a disciplined vocabulary for naming what an agent that acts on real systems *owes*: to know before it acts, to verify what it is told, to treat a capability it can misuse as a trust. That vocabulary turns out to map cleanly onto architectural decisions, and we let it guide the design while keeping every technical claim standing on its own merits.

2 The root cause, formally

It is tempting to describe an agent's memory or learning problems as things that will improve with scale or a cleverer prompt. That framing is wrong, and being precise about why is the whole foundation of the design. Consider what one turn of an LLM-based agent actually computes. Let the input be a token sequence x drawn from a context window of bounded size W . The model applies fixed parameters θ and produces a distribution over the next token; sampling and repeating yields the output y . Abstractly:

$$y = f_{\theta}(x), \quad x \in \mathbb{X}^{\leq W}, \quad \theta \text{ fixed for all inferences.} \quad (1)$$

Three properties of Equation (1) are the source of everything that follows. They are not bugs; they are what a deployed transformer *is*.

(P1) Statelessness. The function has no argument that carries information from one call to the next except x itself. Whatever the agent “knew” during turn t is gone at turn $t+1$ unless it was serialized back into the token string. There is no hidden variable s_t that persists; formally, the map is memoryless: y_t depends on x_t alone, not on the history $(x_1, \dots, x_{t-1})$.

(P2) Frozen parameters. θ does not change as a function of what happens at inference. There is no term in Equation (1) of the form $\theta \leftarrow \theta - \eta \nabla L$: the gradient step that constitutes *learning* happens only in an offline training loop the deployed agent never enters. An outcome in session N — a test that failed, an edit the developer reverted — therefore has no path by which it can alter behavior in session $N+1$.

(P3) Bounded, undifferentiated context. x must fit in W tokens, and every fact competes for the same space on equal terms. There is no separate, larger store the model can address; “give it more context” is bounded by W and, even within W , provides no structure, no provenance, and no persistence.

Now watch the five faculties fall out of P1–P3 by deduction, not observation:

Faculty	Why it is <i>structurally</i> absent	Follows from
Memory (across sessions)	By P1, nothing survives a turn but the token string; by P3, the string is bounded and lost at session end. There is no addressable store that outlives x .	P1, P3
Learning (from outcomes)	By P2, no inference-time event writes to θ . In-context “learning” is real optimization ^{14,15} but lives only inside the current x and vanishes with it (P1) — a <i>simulation</i> of learning, not learning.	P2, P1
Imagination (simulate before acting)	Equation (1) maps tokens to tokens. There is no separate forward model of “what happens to the world (or	P1

Faculty	Why it is <i>structurally</i> absent	Follows from
	codebase) if I take action a ” distinct from emitting more tokens; the model cannot roll out and score a hypothetical it does not also have to narrate.	
Self-correction	Any “check” the model runs is another evaluation of the same f_θ with the same blind spots. There is no independent verifier inside Equation (1); the literature confirms intrinsic self-correction is unreliable without an external signal. ²¹	P2
Impact-awareness	By P3, the model sees only the tokens in x . A million-line repository does not fit; therefore it cannot know, unaided, what elsewhere depends on the symbol it is about to change.	P3

WHY PROMPTING AND TOOLS DO NOT CLOSE THE GAP

A better prompt changes x . More tools (skills, MCP servers, function calls) let the agent fetch new x or emit richer y . Both operate *inside* Equation (1) and leave P1–P3 untouched: the composed system is still a stateless map with frozen weights and a bounded window. A tool call retrieves a document into context, but nothing decides what was worth keeping, consolidates it, or updates the agent's priors for next time. The deficits are properties of the *loop shape* — open, memoryless, one-directional — not of the model's knowledge. To remove them you must change the shape of the loop, which is precisely what an external substrate can do while θ stays frozen.

This reframing is the paper's pivot. If the deficits came from the loop shape, then the remedy is to *re-wrap the loop*: keep f_θ exactly as it is, and surround it with state and update so that the composite system is no longer memoryless, no longer open, and no longer blind beyond W . Figure 1 states the whole thesis in one picture.

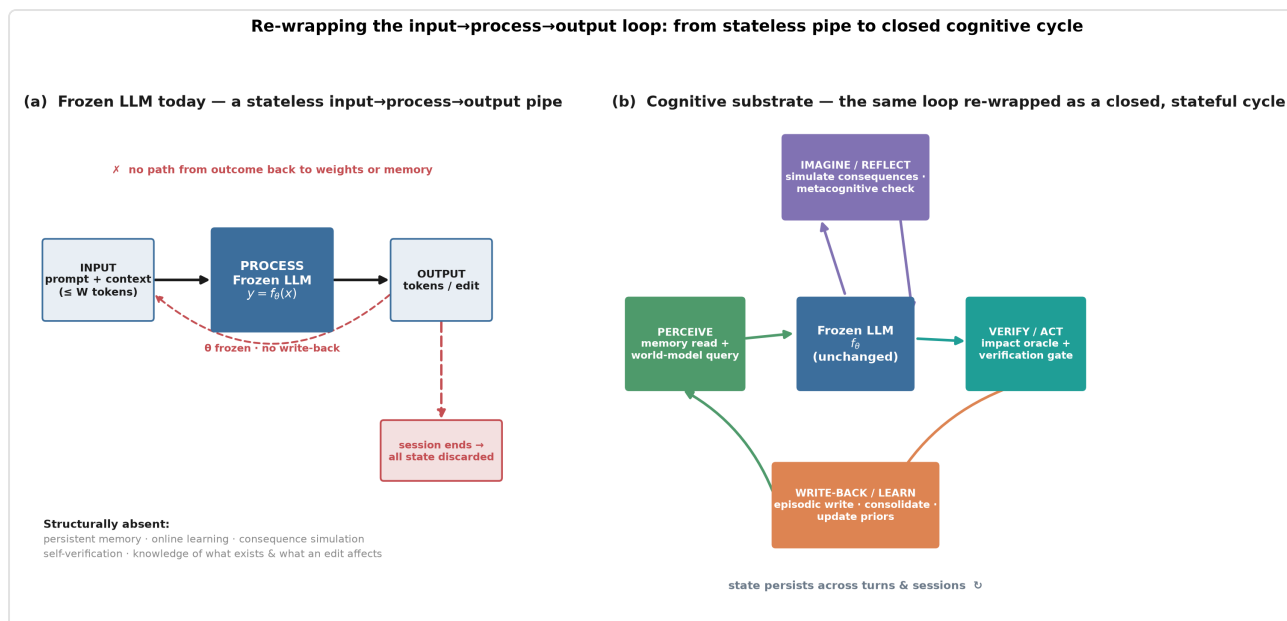


Figure 1. Re-wrapping the input→process→output loop. (a) Today: input enters a bounded window, the frozen model maps it to output, and at session end all state is discarded — there is no path from an outcome back to weights or memory. **(b)** The proposed substrate keeps the identical frozen model (same blue block) but closes the loop into a cycle — perceive (read memory + world-model), reason, imagine/reflect, verify/act, then write back and consolidate — so state persists across turns and sessions. Nothing inside the model changes; the shape around it does.

3 Five faculties and their gaps

Before designing anything, we asked what the research literature has already built for each faculty, and — precisely — what it does not yet give a coding agent. This section is deliberately not a survey for its own sake: for each faculty we state the structural lack (from §2), the best existing approaches with citations, and the **residual gap** that the substrate must actually close. The discipline throughout is to separate what is *solved* from what is merely *named*. All 32 sources are listed in the [references](#); the full structured gap-map is a companion artifact.

MEMORY Persistent experience across sessions

The lack. A transformer's only state is its weights (frozen) and its context window (bounded, discarded at session end); every apparent memory is either baked into pretraining or re-supplied as tokens.

What exists. The idea of an explicit, addressable external memory read and written by a neural controller goes back to Memory Networks¹ and End-to-End Memory Networks², and to the Neural Turing Machine³ and Differentiable Neural Computer⁴, which showed a network can learn to read/write an external matrix via differentiable addressing. But those memories are trained *jointly* with the model and cannot be bolted onto an already-frozen LLM. The pattern that *can* attach to a frozen model is Retrieval-Augmented Generation⁵: pair the generator with a retriever over an external index. For deployed agents, MemGPT⁶ adds an OS-style paging scheme (the model issues calls to move information between context and external storage), and Generative Agents⁷ add a timestamped memory stream with periodic reflection that synthesizes higher-level memories.

Residual gap. None of these gives a coding agent a memory that is simultaneously (a) persistent across sessions without retraining, (b) structured enough to answer “have I seen this bug/pattern before and what happened,” (c) governed by a principled *forget/consolidate* policy rather than unbounded growth, and (d) tied to *verifiable* software artifacts (commits, test outcomes, diffs) rather than free-text the model wrote about itself. RAG is stateless per-query lookup; MemGPT and Generative Agents self-manage paging and reflection with *no ground-truth check* on what is kept or discarded. This is a genuine research gap, not just integration.

LEARNING Durable update from outcomes

The lack. By P2, nothing at inference writes to θ ; in-session adaptation is prompt conditioning that vanishes when context clears.

What exists. Neuroscience offers the shape of an answer: Complementary Learning Systems theory^{8,9} argues intelligence needs *two* systems — a fast, instance-based (hippocampal) learner and a slow, structured (neocortical) one — because a single fast learner catastrophically overwrites old knowledge. On the engineering side, Elastic Weight Consolidation¹⁰ directly mitigates catastrophic forgetting by protecting parameters important to earlier tasks; LoRA¹³ makes cheap, modular weight updates feasible by freezing the base and training small low-rank adapters; Test-Time Training¹¹ shows weights *can* be adapted per-input via a self-supervised loss; Fast Weights¹² give a genuine intermediate timescale of adaptation. And the theory of in-context learning^{14,15} shows prompting can implement implicit gradient descent — real optimization, but session-local.

Residual gap. There is no accepted, low-cost, always-on pathway that takes a concrete outcome (a test failure, a human revert, a review comment) and durably changes future behavior — either by writing to persistent non-parametric state or by cheaply updating parameters — *without* a full offline retraining cycle and *without* catastrophic forgetting. Existing mechanisms are either real-but-deliberate retraining (EWC, LoRA, decoupled from the interaction loop) or real-but-ephemeral in-context simulation. Bridging them into a fast, safe, incremental loop is unsolved.

IMAGINATION

Simulating consequences before acting

The lack. Equation (1) is token→token; there is no forward model of action consequences and no rollout loop.

What exists. Model-based RL has built exactly this — for other domains. World Models²² learn a compressed generative model of an environment and train a controller inside the “dream.” MuZero²³ plans with a learned latent model of only the quantities relevant to planning (reward, value, policy). Dreamer/DreamerV3^{24,25} learn behaviors purely from imagined latent rollouts across many domains. The free-energy principle / active inference²⁶ gives the deepest theoretical grounding for why an agent must predict the sensory consequences of hypothetical actions and act to minimize expected surprise.

Residual gap. Every one of these targets a *continuous perception-action* domain (pixels, physics, game boards) with a learnable dynamics model and a dense reward. None target the *discrete, symbolic* domain of source code, where the dynamics to imagine are “which call sites break,” “does this still type-check,” “do the tests still pass.” A coding agent's imagination faculty needs a fast surrogate simulator over code-change consequences — static analysis, symbolic execution, or an approximate learned model of compile/test outcomes — and no general, reusable such component exists. This is an open build target.

SELF-CORRECTION

An independent check on one's own output

The lack. Any check the model runs re-evaluates the same frozen weights that produced the error.

What exists. Reflexion¹⁸ has the agent verbally reflect on task feedback and store it to condition the next attempt; Self-Refine¹⁹ alternates generator and critic roles on the

same model; Self-Consistency²⁰ marginalizes over many sampled reasoning paths. Crucially, trained *verifiers*¹⁶ and process-level reward models¹⁷ decouple checking from generating by using a separately trained model — escaping the same-weights blind spot.

THE HONEST NEGATIVE RESULT

Huang et al.²¹ evaluated *intrinsic* self-correction — no external feedback — and found it frequently makes correct answers *worse*. The approaches that *do* work (Reflexion, verifiers, process reward models) all inject something the frozen transformer lacks on its own: an external oracle (unit tests, human labels, a separately trained reward model). The lesson for our design is unambiguous: self-correction must be built as an *external verification signal*, not more self-prompting.

Residual gap. For code specifically, no existing system supplies a cheap, general, always-available correctness oracle equivalent to a proof checker: tests exist for some repos but are incomplete, and an LLM-based verifier trained once cannot track an evolving, project-specific notion of correctness. Building that external, evolving check is the open problem — and it is *partly* addressable now, because two concrete external signals already exist for code: the type/compile system and the test suite. Our prototype leans on exactly those.

IMPACT-AWARENESS **Knowing what exists and what an edit affects**

The lack. By P3 the model sees only tokens in x ; a whole repository does not fit, so it cannot know what depends on a symbol it changes.

What exists — and this is the key observation. The tools to compute exact structural impact are *mature and decades old*. The Program Dependence Graph³⁰ (1987) combines control and data dependence in one structure; interprocedural slicing²⁸ (1990) computes what code affects, or is affected by, a given point across function boundaries; program slicing was surveyed as a mature toolbox by 1994²⁹. The Code Property Graph³¹ merges AST, control-flow, and dependence into one queryable graph. On the learned side, “Learning to Represent Programs with Graphs”²⁷ and code2vec³² let neural models see code *structure*, not just token sequences.

THE GAP HERE IS ENGINEERING, NOT SCIENCE

None of these exact tools are wired into the loop of an LLM coding agent. Today's agents either re-read raw file text inside a bounded window or rely on the model's own *unverified guess* about what a change affects. The residual gap is a missing *bridge*: (a) maintain a live, incrementally-updated structural graph of the whole repository, (b) let the agent query “what depends on this symbol” as a fast, sound lookup *before* proposing an edit, and (c) feed the result back into context in a form the model reliably uses. Because this gap is engineering-shaped rather than an open research question, it is the faculty we can honestly *build and evaluate* in one sitting — which is exactly what §8 does.

The pattern across all five is worth stating plainly. Memory, learning, and imagination for code are **genuine research gaps**; self-correction is **partly solved** given an external oracle; impact-awareness is **an integration gap over mature tools**. A credible substrate therefore does not claim to solve all five — it provides the *architecture* into which solved components drop in now and open ones drop in as they mature, and it demonstrates the faculty that is buildable today.

4 The evidence: what the field actually measures

The first edition of this paper argued the five deficits from first principles — from the *shape* of a frozen model, before looking at a single usage statistic. That argument stands on its own. But a design study that only reasons from first principles invites a fair question: *does the field's own evidence agree that these are the real problems?* This edition answers it. We took the load-bearing pain-point statistics that circulate in the 2026 discourse and **independently re-grounded each one from its primary source** — because the project's governing discipline is that a probabilistically-generated number, repeated often enough to sound settled, is still an assumption until you have seen where it comes from.

How we grade evidence — and why it matters here. Each claim below carries a status: **confirmed** (traceable to an independent primary source — a

peer-reviewed paper, the original survey, an official announcement), **vendor-reported** (real and sourced, but only to a company with a commercial stake in the finding, not independently reproduced), or **unverifiable** (we could not corroborate it in primary form). This is the paper's own thesis applied to itself: we do not ask you to trust a figure because it is widely repeated. The re-grounding *changed* what we are willing to assert — and caught three numbers that turned out to be misattributed.

4.1 What the strongest evidence confirms

Five findings survived re-grounding to an independent primary source, and they are enough to carry the argument. The most important is not a vendor's dashboard metric but a **randomized controlled trial**. METR studied sixteen experienced open-source developers across 246 tasks in mature repositories they knew well; the developers *forecast* a 24 % speedup, *believed afterward* they had worked about 20 % faster, and were in fact **19 % slower** with early-2025 AI tools [C1 confirmed · arXiv:2507.09089] . That single result is the empirical heart of this paper: the gap between the *felt* productivity and the *measured* productivity is exactly the gap a system opens when it has no calibrated sense of its own uncertainty (mechanism M2 below), and it is why “the model felt confident” is not evidence of anything.

The trend evidence is equally well-sourced. Stack Overflow's 2025 survey of more than 49,000 developers records trust in AI accuracy **falling from 40 % to 29 %** even as adoption rose to 84 %, with the top-ranked frustration — cited by **66 %** — being code that is “*almost right, but not quite*”, which the survey ties directly to the second-ranked frustration, that debugging AI code is more time-consuming (45 %) [C2 confirmed] . Google's DORA program (~5,000 professionals) frames AI as an “**amplifier**” — magnifying the strengths of strong teams and the dysfunctions of weak ones — and still finds it *increasing delivery instability* at near-universal adoption [C5 confirmed] . On the trustworthiness of the benchmarks themselves, OpenAI **retired SWE-bench Verified** in 2026 after auditing found at least **59.4 % of the hard problems it examined had flawed test cases** or training-data contamination [C6 confirmed] — a caution that applies squarely to our own prototype's numbers, and one reason we report them the way we do. And the reason an agent *cannot* be its own sole judge is not rhetorical: Panickssery et al. (NeurIPS 2024, Oral) show LLM evaluators **recognize and**

systematically favor their own generations [C12 confirmed · arXiv:2404.13076] — the empirical foundation for insisting that verification be *external*.

These five are enough. A randomized trial (miscalibration), a 49,000-person survey (“almost right”), a 5,000-person study (instability), a benchmark retirement (don't trust the metrics), and a peer-reviewed result (an LLM favors its own output). Each maps onto a specific deficit this paper's architecture supplies — and none depends on a vendor's self-report.

4.2 What is real but vendor-reported — and what did not survive

Several widely-cited figures are *real and sourced* but trace only to a company selling a remedy for the problem the figure describes; we use them as corroboration, not proof, and we say so. Veracode reports **45 % of AI-generated samples introduce an OWASP-Top-10 vulnerability** [C3 vendor] ; Faros' telemetry across 22,000 developers reports median PR-review time up **441.5 %** and **31.3 % of PRs merged with no review** [C7 vendor] ; Sonar reports the sharpest single number in the whole discourse — **96 % of developers do not fully trust AI code, yet only 48 % always verify it**, a 48-point “*verification gap*” [C9 vendor] . These are consistent with the confirmed evidence and with each other, which is why we cite them — but a reader should weigh them as vendor telemetry, not as independent science.

The discipline earned its keep on three claims that **did not survive**, and which this paper therefore *does not repeat as fact*:

- The widely-quoted “**2.74x more vulnerabilities**” figure could not be traced to Veracode's own report; it appears to be conflated with a separate study. Only the 45 % figure is genuinely Veracode's. We drop the multiplier.
- The claim that “**Anthropic's 400,000-session study found 17 % lower comprehension**” merges two different studies — the session study contains no comprehension finding. We do not use the number.
- GitClear's code-duplication rise is quoted as both **4x and 8x** in the vendor's own title versus body, and JetBrains' “77 % correct conventions every session” could not be located in the primary report at all. We cite the direction (duplication is rising) without the disputed multiplier.

That a re-grounding pass changed our own claims is the point, not an embarrassment. It is the same move the architecture in §7 makes structural: *a stored fact is provisional until an independent check confirms it*. A full source-by-source table — every claim, its primary citation, and its status — is in the **Evidence appendix**. The mechanisms that follow are each motivated by a *confirmed* finding wherever one exists, and lean on vendor evidence only where that is the best the field currently offers.

5 The Qur'anic epistemic lens

HOW THIS LENS IS USED — AND HOW IT IS NOT

The Qur'an is used here as a **framing lens and ethics source**, never as technical authority for an engineering claim. No verse is cited to prove that an algorithm works or that a data structure is correct — those claims stand on their engineering merits alone (§3, §8). What the lens supplies is threefold: (1) a precise *vocabulary of obligation* for what an agent that acts on real systems owes — to truthfulness, to verification, to stewardship; (2) a *hierarchy of knowledge* (‘ilm → fahm → hikma: knowledge → understanding → wisdom) that motivates a *layered* memory architecture rather than a flat vector store; and (3) *ethical constraints on autonomy* that translate into concrete safeguards. Where a mapping is marked *load-bearing*, the concept motivates a specific design decision (e.g. a *mandatory*, not optional, verification gate); where marked *metaphor*, it is illustrative. Canonical text below is presented directly and attributed; it is not paraphrased. Arabic and translations were retrieved from quran.ai; the full 14-row mapping table is in the [appendix](#).

The lens earns its place because the deepest failure modes of an autonomous coding agent are not computational but *epistemic and ethical*: acting without knowing, trusting a report without checking it, and treating a granted capability as license. The Qur'anic vocabulary names these with unusual precision, and three verses in particular map so directly onto architectural decisions that they shaped the design rather than decorating it.

17:36 — LĀ TAQFU · THE ROOT OF IMPACT-AWARENESS [LOAD-BEARING]

وَلَا تَقْفُ مَا لَيْسَ لَكَ بِهِ عِلْمٌ إِنَّ السَّمْعَ وَالْبَصَرَ وَالْفُؤَادَ كُلُّ أُولَٰئِكَ كَانَ عَنْهُ مَسْئُولًا

“Do not follow blindly what you do not know to be true: ears, eyes, and heart, you will be questioned about all these.”

→ **Design principle.** Before any mutation (write, delete, refactor), a pre-action gate must establish what the agent actually knows: which entities the change affects, whether the relevant files/tests/dependents were actually read, and whether the predicted outcome rests on evidence rather than a pattern-matched guess. Actions taken without verified knowledge are *blocked*, not merely flagged. The verse's own structure — “ears, eyes, heart... questioned about all these” — maps to an **audit trail**: every channel the agent used (what it read, inferred, assumed) is logged so the decision can be reconstructed and questioned. This is precisely the impact oracle of §8.

Grounded with quran.ai: `fetch_translation(17:36, en-abdel-haleem);`
`fetch_tafsir(17:36, en-ibn-kathir)`

49:6 — TABAYYUN · THE VERIFICATION GATE [LOAD-BEARING]

يَا أَيُّهَا الَّذِينَ آمَنُوا إِن جَاءَكُمْ فَاسِقٌ بِنَبَأٍ فَتَبَيَّنُوا أَن تُصِيبُوا قَوْمًا بِجَهَالَةٍ فَتُصْحَبُوا عَلَىٰ مَا فَعَلْتُمْ نَادِمِينَ

“Believers, if a troublemaker brings you news, check it first, in case you wrong others unwittingly and later regret what you have done.”

→ **Design principle.** The architecture places a verification step between *receiving information* (from context, tool output, or its own prior reasoning) and *acting on it*. The operative term *tabayyun* demands active investigation, not passive acceptance: before applying a fix based on an error report or its own diagnosis, the agent re-reads the current file state, confirms the issue still exists, and checks the fix introduces no new breakage detectable by types or tests. The gate is *architectural and mandatory* — the verse's command is categorical, not conditional on the reporter's trustworthiness — which is exactly the design lesson the self-correction literature reached empirically²¹: a real external check, not more self-prompting.

Grounded with quran.ai: `fetch_translation(49:6, en-abdel-haleem);`
`fetch_tafsir(49:6, en-ibn-kathir)`

2:31–32 — TA'LĪM AL-ASMĀ' · THE WORLD-MODEL [LOAD-BEARING]

وَعَلَّمَ آدَمَ الْأَسْمَاءَ كُلَّهَا... قَالُوا سُبْحَانَكَ لَا عِلْمَ لَنَا إِلَّا مَا عَلَّمْتَنَا

“He taught Adam all the names [of things]... They said, ‘May You be glorified! We have knowledge only of what You have taught us.’”

→ **Design principle.** Knowledge begins with *naming* — identifying entities and their relations. The agent must hold a structured map of what exists in the codebase (files, functions, classes, dependencies), not a flat listing: knowing that *A* calls *B*, that module *X* depends on *Y*, that test *T* covers class *C*. The angels' admission — “we have knowledge only of what You have taught us” — is a startlingly exact description of the LLM's own situation under P3: it knows only what is in its window. The external world-model supplies the “names” of entities that exceed context capacity.

Grounded with quran.ai: `fetch_translation(2:31-32, en-abdel-haleem)`

33:72 — AL-AMĀNA · STEWARDSHIP & BOUNDED AUTONOMY [LOAD-BEARING]

إِنَّا عَرَضْنَا الْأَمَانَةَ عَلَى السَّمَاوَاتِ وَالْأَرْضِ وَالْجِبَالِ فَأَبَيْنَ أَنْ يَحْمِلْنَهَا... وَحَمَلَهَا الْإِنْسَانُ
إِنَّهُ كَانَ ظَلُومًا جَهُولًا

“We offered the Trust to the heavens, the earth, and the mountains, yet they refused to undertake it and were afraid of it; mankind undertook it — they have always been inept and foolish.”

→ **Design principle.** An agent that can modify a codebase bears an *amāna* — accepted responsibility for something it can damage. The verse's structure is the design: the heavens declined the trust, recognizing its weight; the human bore it and is called *ẓalūman jahūlā* (given to wrong and ignorance). So the architecture (a) operates under **least privilege** — a granted capability is never blanket permission; and (b) *assumes the agent will err* and builds in **reversibility** (sandboxing, staged commits, rollback), **audit**, and **scope bounds** as structural safeguards. The trust is not “the agent is trustworthy”; it is “the agent has accepted accountability for a domain it can harm, and the architecture must respect that weight.” This is the governance boundary enclosing the entire system in Figure 2.

Grounded with quran.ai: `fetch_translation(33:72, en-abdel-haleem);`
`fetch_tafsir(33:72, en-ibn-kathir)`

Four further anchors complete the mapping (full text in the [appendix](#)). **20:114** — “*rabbī zidnī ‘ilmā*,” “My Lord, increase me in knowledge” — frames knowledge as perpetually incomplete and growing, motivating the continual-learning store that accumulates across sessions. **96:1–5** — *iqra’* and the teaching “by the pen” (*al-qalam*) — is the principle of **externalized memory**: the pen turns ephemeral thought into durable record, exactly the write-back mechanism a bounded context window requires. **4:82** and **47:24** — *tadabbur*, deep reflection, whose root *d-b-r* concerns “what comes after” / consequences — ground the metacognitive controller as a structured *trace-forward* through consequences (“if I apply this, what breaks?”), not a vague confidence score. And the classical epistemic ladder **‘ilm** → **fahm** → **hikma** motivates the three-layer memory of §7: raw logs, consolidated patterns, decision support — dumping everything into one flat store collapses the hierarchy and loses the distinction between fact and actionable understanding.

The remarkable thing is not that these mappings are poetic; it is that they are *operational*. “Verify before acting” is not a sentiment here — it is a mandatory gate in the action pipeline. “Know the names of things” is not a metaphor — it is a dependency graph. The lens told us *which* safeguards are non-negotiable; the engineering told us how to build them.

6 Six mechanisms the frozen loop also lacks

The five faculties of the first edition answer “what cognitive capabilities does a stateless model structurally lack?” This edition adds a second family of deficits that the field's evidence forced into view — not *cognitive* faculties but **metacognitive and resource-allocation** ones: knowing how much effort a task deserves, knowing what you have not been told, knowing when to split work, when you have drifted, when you have over-built, and when to check. A frozen map $y = f_{\theta}(x)$ lacks these for the same reason it lacks the first five: each requires *state* the model does not carry and a *decision about the computation itself* that a single forward pass cannot make. We formalize six.

A note on honesty before we start. Two of these six are *already largely solved* by existing tooling, and we say which (M1, M3). Naming a mechanism is not

claiming to have invented it. The contribution of this section is the *unified account* — showing that all six fall out of the same frozen-loop shape, and that the genuinely unsolved ones (M2, M5) are unsolved for a structural reason, not for lack of effort.

5.1 M1 — Complexity-aware routing

The deficit. A frozen model has no notion of its own running cost. The same f_{θ} is invoked whether the task is `is_prime(n)` or a distributed rate-limiter, and a premium model billed per token spends the same premium rate on both. The project stated it plainly: “a simple prime-number finder — if you use [a premium model] it will not give you extra.” **Formally.** let a task x have an intrinsic complexity $c(x)$ and let a tier ladder $T_1 < \dots < T_k$ have costs $\kappa_1 < \dots < \kappa_k$. Routing selects the *cheapest* tier whose capability covers the task:

$$\text{route}(x) = \min\{ T_i : \text{capable}(T_i, c(x)) \}, \quad \text{then escalate } T_i \rightarrow T_{i+1} \text{ only if an external check on the output fails.}$$

The escalation clause is what makes this safe: the worst case is a cheap attempt plus a premium attempt, the common case is cheap alone, and the decision to spend more is driven by a *verified failure*, never by the model's self-assessment. **Ecosystem status: largely solved.** Model tiering and gateways (LiteLLM, OpenRouter, per-agent `model:` fields) already route by cost. What they do *not* give is a *transparent, per-task, auditable* complexity judgment the user can see and override before dispatch — that thin transparency layer is the only thing left to build, and our prototype (§9) builds it. Anchor: 17:36, “do not pursue what you have no knowledge of” — spend capability in proportion to what the task actually requires.

5.2 M2 — The assumption / uncertainty gate root failure

The deficit. This is the failure the project named as central: “the biggest problem is *Assumption*. If it doesn't have enough context it will assume many things.” A model that computes $\arg\max_y P(y \mid x)$ always returns *some* most-likely continuation, even when x under-determines the task — there is no term in the objective that

fires when the input is *insufficient*. The model cannot distinguish “I know this” from “this is merely the least-improbable guess.” This is precisely what the METR trial measured from the outside: confident forecasts, slower reality [C1 confirmed]. **Formally**, define a specification-completeness functional $s(x) \in [0,1]$ over the dimensions a task needs pinned down (inputs and outputs, target scope, success criteria). The gate interposes *before* execution:

if $s(x) < \tau$: **halt** and emit the missing-information questions | else : proceed to route(x).

The gate spends *zero* generation tokens on an under-specified request. Asking one question is cheaper than the “almost right, but not quite” rework loop that 66 % of developers report [C2 confirmed]. **Ecosystem status: residual gap — genuine whitespace.** Nothing in the 2026 stack supplies calibrated “known-unknowns”: the pain-point survey names it directly — “*models rarely signal uncertainty or say ‘I can’t do this’*” — and spec-driven-development tools only help when the human already wrote a complete spec, which is the assumption the gate is meant to remove. Our prototype builds this too. Anchor: 49:6, *tabayyun* — “if a source brings you news, verify it”: the discipline of not acting on an unverified report is exactly a gate on insufficient input.

5.3 M3 — Task / session decomposition

The deficit. A bounded, undifferentiated window (property P3) means a long session accretes unrelated sub-tasks that compete for attention and degrade retrieval. The project’s remedy was exact: “*instead of using multiple sessions for multiple independent tasks... we normally do full things in a single session.*” **Formally**, given a task set with a dependency relation, the decomposition problem is to partition into contexts

$C_1, \dots, C_m$ that minimize cross-context coupling while keeping each context’s working set inside the window — independent components run in isolated sessions, dependent ones share state. **Ecosystem status: solved.** Subagents with isolated windows, the experimental Agent-Teams pattern, and git-worktree fan-out already implement this well; the only residue is that *choosing the partition boundary* is still a human heuristic. We flag it and move on — a paper that claimed to reinvent subagents would be exactly the over-selling this project forbids. Anchor: 20:114, “do not hasten...” — ordered, separated pursuit over one overloaded pass.

5.4 M4 — Goal-anchoring against drift

The deficit. To a text model, “*generation of a long story and code is the same concept*” — both are just high-probability token sequences — so nothing intrinsic keeps output tethered to the *objective* rather than to local fluency. The project: the agent “*diverts from the main goal.*” **Formally**, let g be the goal representation fixed at $t=0$ and y_t the work in progress; drift is a rising divergence $D(y_t, g)$. Anchoring adds a periodic check that re-validates y_t against g and corrects when D exceeds a bound — the goal is *re-read*, not assumed to still be in view. **Ecosystem status: partial.** CLAUDE.md and /goal hold a static anchor, but the field documents it decaying within a session (“context compression wiping scrollbar”), and the open gap is that “*specs drift out of sync with code*” with no tool continuously re-verifying the two. Anchor: the *ḥifẓ / murāja’a* discipline — retention through active review, not one-time loading.

5.5 M5 — Anti-over-engineering residual gap

The deficit. The maximum-likelihood objective favors the *elaborate* completion: more abstraction, more defensive scaffolding, more “production-ready” ceremony than the task asked for. The project named the cost precisely: “*the default version will be over-engineering — waste of tokens + time + quality and cost.*” **Formally**, let a solution have footprint $\varphi(y)$ (files touched, abstractions introduced, lines added) and let the task imply a minimal sufficient footprint $\varphi^*(x)$. Over-engineering is $\varphi(y) - \varphi^*(x) > 0$; the check flags scope the task did not request. **Ecosystem status: residual gap.** The one discipline that exists — the frontend-design skill’s minimalism rule — is explicitly UI-only; no general tool measures unnecessary abstraction against the stated task, and the failure is invisible to the review agents that scan for bugs. Anchor: 7:31, “...do not be excessive” — sufficiency as a virtue; the least structure that meets the need.

5.6 M6 — Inline verification

The deficit. A human “*interprets logic while writing... its real-time verification*”; a frozen model emits the whole sequence and only afterward can anything check it. Verification is *deferred*, and the field’s sharpest statistic is the size of that deferral: the 48-point gap between the 96% who distrust AI code and the 48% who always verify it [C9 vendor],

with the deferred review time itself up 441.5% [C7 vendor] . And the verifier cannot be the same model, because an LLM favors its own output [C12 confirmed] . **Formally**, interleave an external check $v(\cdot)$ at each meaningful step j rather than once at the end: $y = (y_1, \dots, y_n)$ with $v(y_j)$ gating y_{j+1} — the same external-check principle the substrate uses at the architecture level (§7, the mandatory verify-gate), pushed down to the generation step. **Ecosystem status: partial.** Streaming and PostToolUse hooks give passive visibility and post-edit linting, but no shipped mechanism forces an *interpretive* checkpoint *during* generation. Anchor: 49:6 again, and 17:36 — verification as a continuous obligation, not a final formality.

The pattern across all six. Each mechanism is a *decision about the computation* that a single forward pass cannot make: how much to spend (M1), whether to proceed (M2), how to split (M3), whether you have drifted (M4), whether you have over-built (M5), whether to check now (M6). None can live *inside* f_θ ; all live in the stateful wrapper around it — which is the same conclusion the five faculties reached, arrived at from the direction of *resource and control* rather than *cognition*.

7 The cognitive substrate

We now specify the architecture. The design rule is strict: **the LLM stays frozen**. We add no term that modifies θ . Everything is external state and external update, composed so that the *system* — not the model — acquires the missing faculties. Where §2 wrote the agent as the memoryless map $y = f_\theta(x)$, the substrate replaces it with a *stateful* operator over a persistent store M :

$$(y_t, M_{t+1}) = F(x_t, M_t; f_\theta) \quad (2)$$

Read Equation (2) against P1–P3: it takes the previous store M_t as an explicit argument (defeating statelessness, P1), it returns an *updated* store M_{t+1} (a write-back path that P2 forbade for weights, now legal because it targets external state), and M can be

arbitrarily larger than the window W (defeating P3). The frozen f_θ appears only as a subroutine. The composite F is what Figure 1(b) draws and what the rest of this section defines.

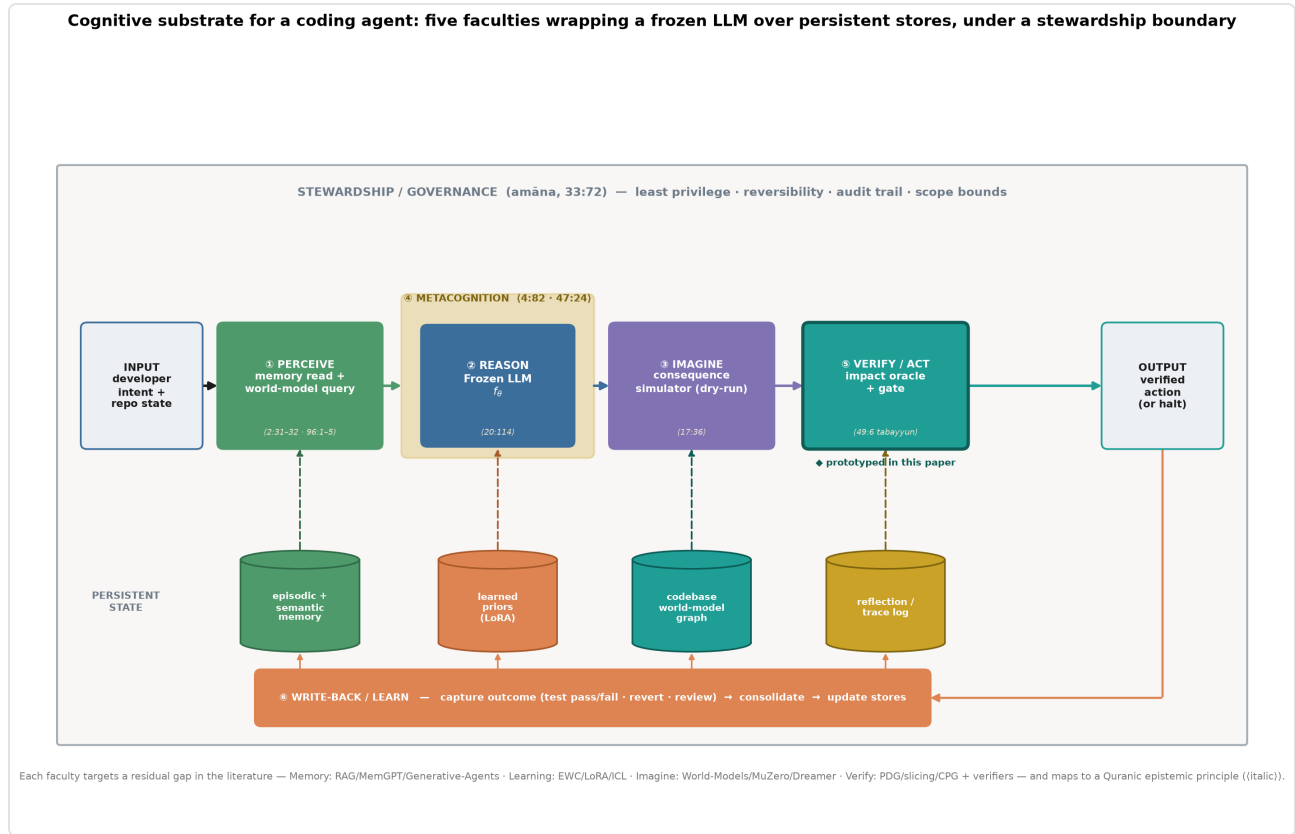


Figure 2. The cognitive substrate for a coding agent. A five-stage faculty pipeline wraps the frozen LLM (“REASON”): ① **perceive** reads memory and queries the world-model; ② **reason** is the unchanged model; ③ **imagine** dry-runs consequences; ④ **metacognition** (the collar) checks self-consistency; ⑤ **verify/act** is the impact oracle plus the mandatory gate. Beneath sit four **persistent stores** (episodic/semantic memory, learned priors, the codebase world-model graph, the reflection log); the ⑥ **write-back/learn** band captures each outcome and consolidates it into those stores. The whole system runs under a **stewardship boundary** (least privilege, reversibility, audit, scope). Each faculty is tagged with its Qur'anic anchor (§5) and targets a residual gap from §3. The verify faculty (heavy border) is the one prototyped in §8.

7.1 The five faculties, formally

Perceive — memory as an addressable store. Let the memory be a growing set of records $M = \{m_i\}$, each an embedding-keyed entry with provenance (a commit, a test outcome, a past edit) and metadata (timestamp, access count, validation status).

Perception retrieves a relevant slice by a score that — following Generative Agents⁷ but

adding a validity term the literature lacks — combines relevance, recency, and *grounded importance*:

$$\text{retrieve}(x, M) = \text{top-}k_i \sigma(\alpha \cdot \text{rel}(x, m_i) + \beta \cdot \text{rec}(m_i) + \gamma \cdot \text{val}(m_i)) \quad (3)$$

The *val* term — did this memory's prediction later prove correct against a test or commit? — is the load-bearing addition: it is what lets memory be pruned by *ground truth* rather than by the model's own say-so, the gap named in §3. This is the ‘ilm layer of the epistemic ladder.

Reason — the frozen model, unchanged. The LLM receives an assembled context: the raw request plus the retrieved memory slice plus the world-model's answer to “what does this touch?” It contributes exactly what it is good at — synthesis, code generation, natural-language understanding — and nothing is asked of it that P1–P3 forbid.

Imagine — a consequence simulator. Define a surrogate dynamics model g that maps a proposed action a and current codebase state C to a predicted outcome without touching the real repository:

$$\hat{c} = g(a, C) \rightarrow \text{predicted } \{ \text{broken call sites, type errors, failing tests} \} \quad (4)$$

Unlike World Models or MuZero^{22,23}, g here need not be learned from pixels or reward: for code it can be *partly exact* — static analysis and the dependency graph give sound over-approximations of “what breaks” — and partly cheap simulation (run the affected tests in a sandbox). This is the faculty the literature has *not* built for the symbolic domain (§3); the impact oracle of §8 is a first, exact-analysis instance of g .

Metacognition — a structured self-consistency pass. Following the honest lesson of §3–§5 (self-critique on the same weights is unreliable²¹; a real external check is required), the controller does not ask the model “are you sure?” It runs a *tadabbur* trace-forward: it checks the proposed action against (i) the model's own stated plan, (ii) the type system, and (iii) the existing test suite — three signals at least two of which are external to f_θ . Inconsistency halts and backtracks rather than proceeds.

Verify/act — the mandatory gate. The *tabayyun* gate (49:6) is a hard predicate on the assembled evidence $E(a)$: the action proceeds only if the impacted set has been computed, the relevant state re-read, and no external check contradicts the prediction.

$$\text{act}(a) \text{ allowed} \Leftrightarrow \text{gate}(E(a)) = \checkmark \wedge \text{scope}(a) \subseteq \text{granted} \quad (5)$$

7.2 Learning without touching θ

The write-back band closes the loop. After an action resolves, its outcome — test pass/fail, a human revert, a review comment — is a *labeled* training signal, the external oracle §3 said learning requires. Two update channels, both leaving θ frozen, are available and are ordered by cost:

1. **Non-parametric (always-on):** write the outcome to M with its provenance, and update the *val* scores of the memories that informed the action. This is instant, reversible, and the primary path — the fast, hippocampal side of Complementary Learning Systems^{8,9}.
2. **Parametric (deliberate, optional):** when a pattern recurs with high validated confidence, distill it into a small LoRA adapter¹³ trained offline — the slow, neocortical side — with EWC-style protection¹⁰ against forgetting. This never runs inside the interaction loop.

The consolidation process between them is the *ḥifẓ + murājaʿa* principle: periodic review that reinforces recurrent high-value patterns, decays stale entries, and resolves contradictions — the forgetting policy whose absence §3 flagged in RAG and MemGPT. It is what keeps M from growing without bound and drowning signal in noise.

WHAT IS GENUINELY LOAD-BEARING HERE

The architecture's one non-obvious commitment is that *learning and memory are anchored to verifiable software artifacts* — commits, type checks, test outcomes — rather than to the model's own summaries of itself. This is the single design choice that separates the substrate from “RAG plus a scratchpad,” and it is possible *only* in domains that have a cheap external oracle. Code is such a domain. That is why a coding agent, and not a general chat agent, is the right first target — and why the faculty we prototype next is the one where that oracle is exact. Figure 3 draws the complete picture: the five faculties of this section governed by the six mechanisms of §6 as a metacognitive control layer.

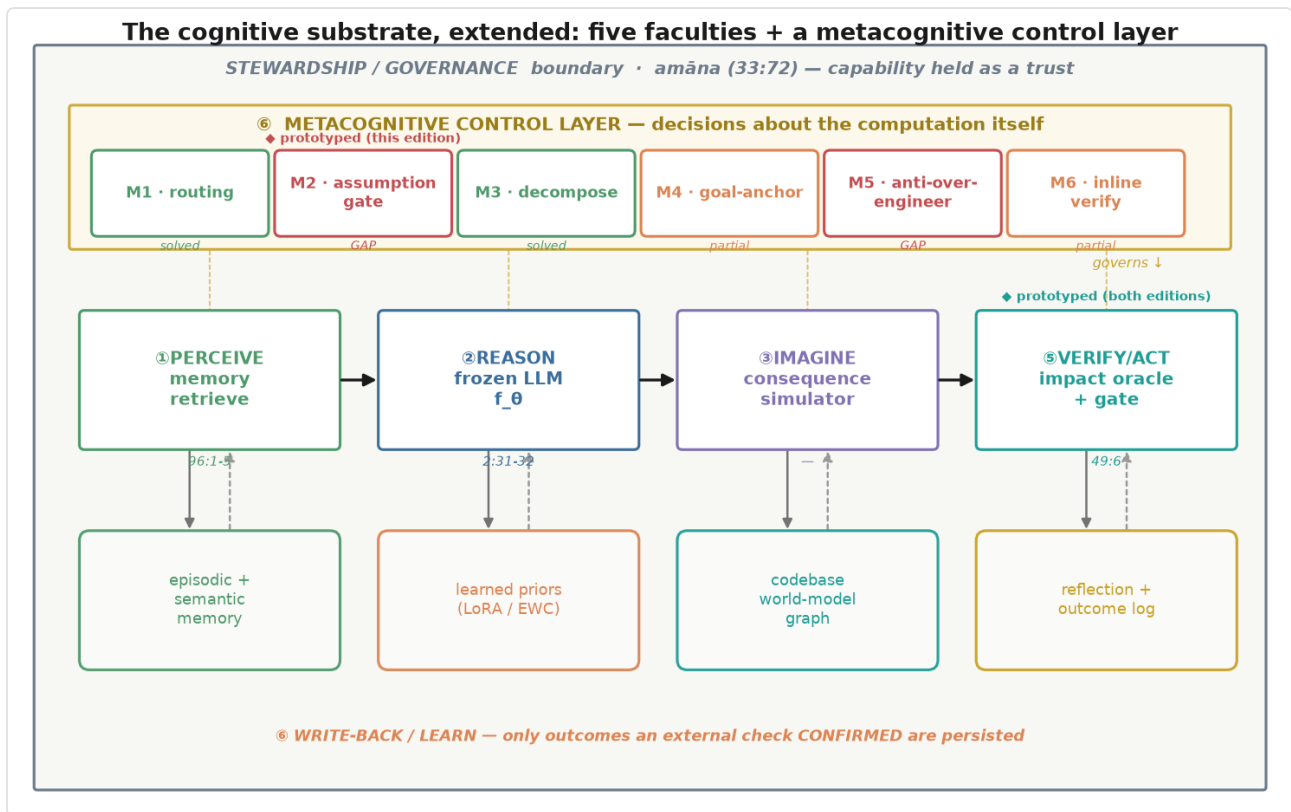


Figure 3. The substrate, extended with the six mechanisms of §6. The five-faculty pipeline of Figure 2 (perceive→reason→imagine→verify/act) is now governed by a **metacognitive control layer** — the six mechanisms that decide *about the computation itself*: how much to spend (M1), whether to proceed (M2), how to split (M3), whether it has drifted (M4), whether it has over-built (M5), when to check (M6). Each is tagged with its ecosystem status — **solved**, **partial**, or **residual gap** — and the two prototyped in this edition (M1 routing, M2 gate) are marked. Beneath sit the same four persistent stores; the whole runs under the stewardship boundary (*amāna*, 33:72).

8 Prototype: a Codebase World-Model & Impact Oracle

Design papers are cheap; the discipline is to build the one faculty where a claim can be *tested* in a single sitting. That faculty is impact-awareness (§3: mature tools, missing bridge). We implemented two of the substrate's components as a runnable Python package: the **Codebase World-Model** (the “perceive” store, 2:31–32) and the **Impact Oracle** (an exact instance of the “imagine→verify” simulator *g*, 17:36). The full source, demo, and tests are the companion artifact `impact_oracle_src.zip`.

8.1 What it does

World-model. A parser walks a repository with Python's `ast` module and builds a directed graph whose nodes are symbols — modules, classes, functions, methods, module-level names — with stable qualified IDs (`pkg.mod.Class.method`) and metadata (file, line, kind, signature). Edges capture five structural relations: `imports` , `calls` , `inherits` , `references` , `contains` . The graph is *persisted* to disk keyed by a per-file content hash, so an unchanged file is never re-parsed — the “persistent memory of what already exists,” incremental by construction.

Impact oracle. Given a proposed change to a symbol X , the oracle traverses the graph along *reverse* dependency edges (who calls / imports / inherits / references X), transitively, assigning each reached node a confidence that decays with hop distance and edge type. The output is a ranked *blast radius* — the set of symbols and files predicted to be affected — each with an explanation path back to X , and a tunable confidence threshold. Figure 4 shows one such propagation.

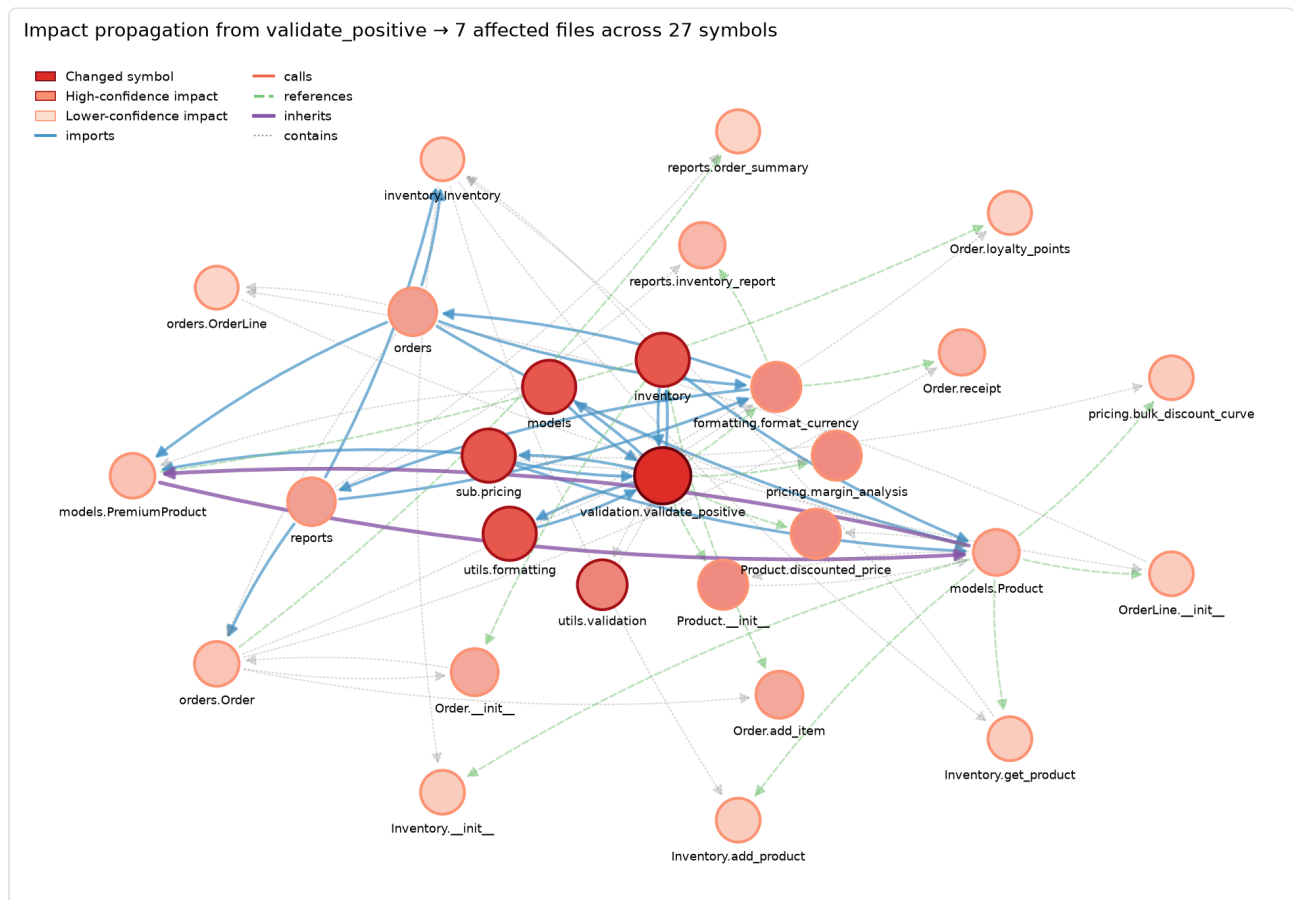


Figure 4. Predicted blast radius of one edit. Changing `utils.validation.validate_positive` (dark red, center) propagates along reverse-dependency edges to 27 symbols across 7 files. Node shade encodes confidence (high → pale by hop distance); edge style encodes relation (calls, references,

inherits, imports, contains). This is the “imagine” faculty made concrete: the consequence set is computed *before* any edit is applied.

8.2 How we evaluated it — against ground truth, not self-grading

The temptation in a prototype is to let the system grade its own homework. We refused that. Ground truth comes from **mutation testing**, which is independent of the graph the oracle uses: we built a realistic 10-module demo package (classes, cross-module imports, inheritance, call chains) with a 36-test `pytest` suite; then for each of five target symbols we *actually mutated the symbol to break its contract* and ran the suite. The set of files whose tests then failed is the *true* behavioral blast radius — observed, not predicted. We compared three predictors against it:

- **Graph Oracle** — our reverse-dependency traversal.
- **Grep baseline** — text-search for the symbol name (what an LLM agent does today when it “greps around”).
- **Edited-file-only** — assume impact stops at the file you edited (the implicit assumption of a narrow-context agent).

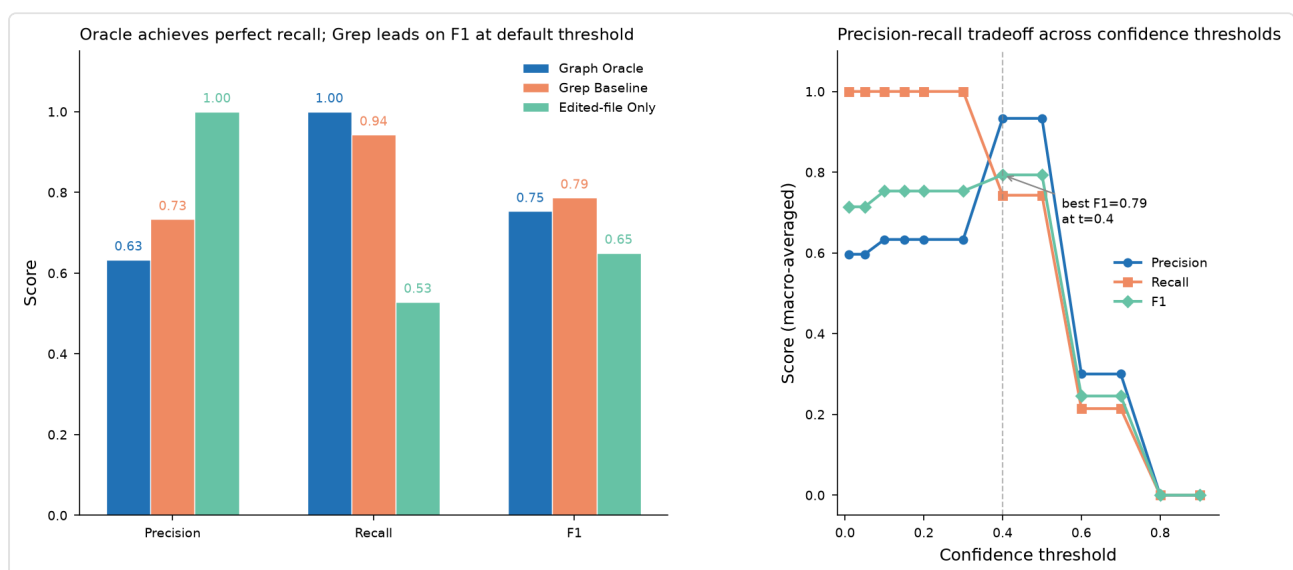
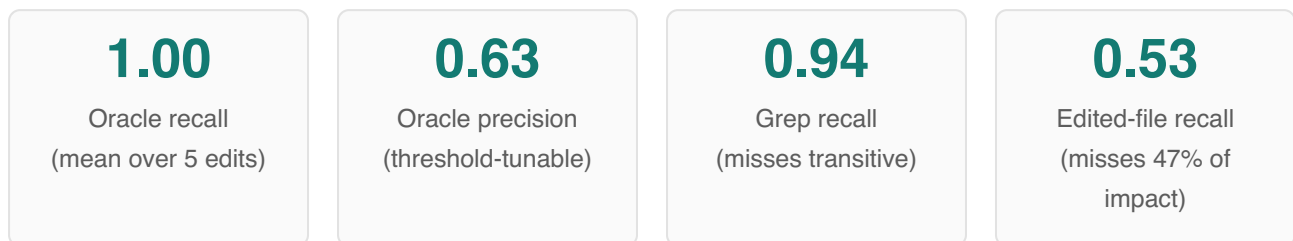


Figure 5. Honest evaluation against mutation-derived ground truth. (left) The oracle is the *only* method with perfect recall (1.00) — it never misses an affected file — at a precision cost (0.63) versus

grep (0.73). Edited-file-only achieves perfect precision but catastrophic recall (0.53), missing nearly half of all real impact. **(right)** Precision and recall trade off with the confidence threshold; best F1 = 0.79 at threshold 0.4. The chart title states the result plainly, including where grep leads on F1 — we do not oversell.

8.3 Reading the result honestly

The oracle does *not* dominate every metric, and the paper is stronger for saying so. On F1 at the default threshold, the grep baseline (0.79) slightly edges the oracle (0.75). What the oracle uniquely provides is **guaranteed recall**: across all five mutations it never once missed a file that actually broke, whereas grep missed transitive dependents (a caller that never mentions the symbol by name) and edited-file-only missed 47% of real impact. For the question that matters when you are about to change production code — “show me everything that could break” — a false negative (a silent breakage) is far more costly than a false positive (an extra file to glance at), and only the structural oracle drives false negatives to zero. Its precision, moreover, is *tunable*: Figure 5(right) shows the operating point can be moved along the curve, where neither text baseline offers such a control.

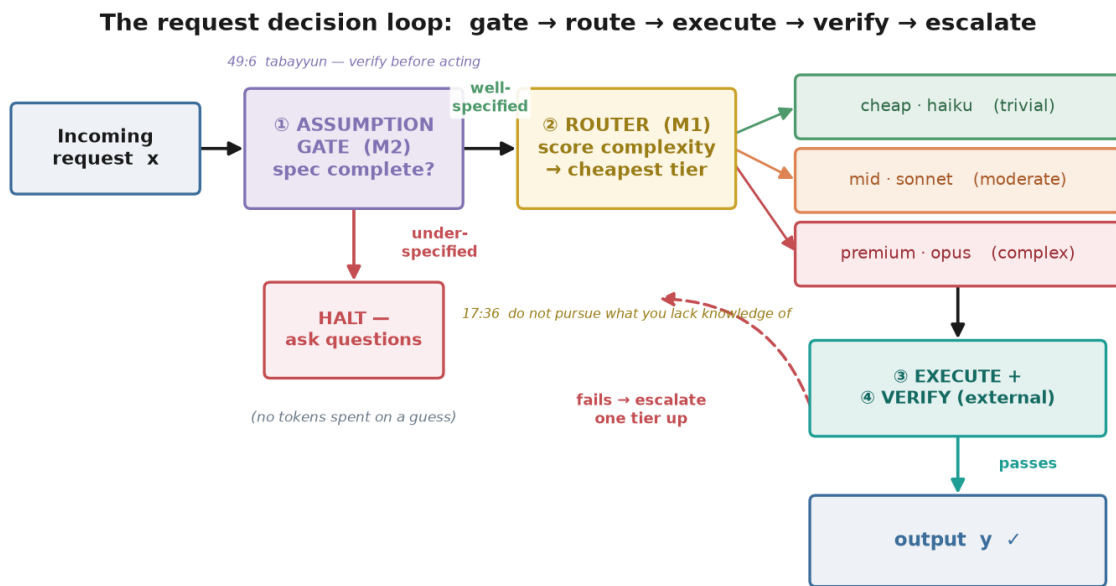
It also scales. Beyond the toy package, the same parser handled real standard-library code: it built a 303-node graph of the `json` package in 18 ms and a 1,903-node graph of the `http` package in 91 ms, answering impact queries in under a millisecond. The approach is not a toy that only works on toys; it is a mature technique that simply had not been placed in the agent's loop.

WHAT THIS PROVES, AND WHAT IT DOES NOT

Proves: the impact-awareness faculty of the substrate is buildable today, runs on a laptop, persists its world-model, and beats the two strategies a context-bounded LLM agent actually uses — on the metric (recall) that governs safety — against independent ground truth. **Does not prove:** that the other four faculties are equally easy (they are not — §3), that this Python-`ast` analysis handles dynamic dispatch, reflection, or cross-language repos (it does not yet — §12), or that the whole substrate has been built end-to-end (it has not). This is one faculty, demonstrated; the rest is specified and argued.

9 Prototype II: a complexity-aware router and an assumption gate

The impact oracle prototyped one faculty. This second prototype builds the top of the build-opportunity map from the other direction — the two mechanisms whose pain the field's evidence supports most directly: **M1 complexity-aware routing** and **M2 the assumption gate**. They compose into a single decision loop, and the whole thing runs live on the real model ladder, so the numbers below are measured, not asserted.



Every decision is a transparent, attributable rubric — not another opaque model call. Trust is earned by the external check (④), never asserted by the model.

Figure 6. The composed loop: an incoming request first meets the *assumption gate*, which halts and asks if the specification is incomplete (M2); a well-specified request is scored by the *router* and sent to the cheapest capable tier (M1); the output is checked by an *external* verifier, and only a verified failure escalates one tier up.

Every decision is a transparent rubric — never another opaque model call.

9.1 The design commitment: transparent rubrics, external trust

The project's discipline — “*never trust AI output; it is mathematically-calculated probability*” — rules out the obvious lazy design. We do **not** ask a second LLM “how hard is this task?” or “is this well specified?”, because that merely moves the untrusted probability up a level and hides it. Instead both the router and the gate are **transparent, additive rubrics over explicit features**. The router scores algorithmic and architectural signals, moderate-complexity markers, multi-step structure, and length, and *subtracts*

for trivial-task markers — every point attributable to a named feature the user can inspect and override. The gate scores *information content*: concreteness anchors (a code fence, a call signature, a worked `input→output` example, a named file), specific named technologies, minus vague fillers and brevity. Neither delegates its judgment to a model. And escalation — the only path that spends premium tokens — is driven by an *external* check on the output, the same principle the substrate uses at the architecture level (the mandatory verify-gate, §7) and M6 uses at the generation step.

9.2 How we evaluated it — live, on real models

We hand-labeled a **30-task set**: eight trivial, seven moderate, six complex (each with a gold tier), and nine deliberately under-specified (gold label: *should ask*). We ran every task through the full loop **live** on the actual ladder — cheap = `claude-haiku-4-5`, mid = `claude-sonnet-5`, premium = `claude-opus-4-8` — measuring *real* token counts from each call. Cost is exact arithmetic on those measured tokens against published per-tier list prices; the always-premium baseline reprices the *identical* tokens at the top tier, so any saving comes purely from tier selection.

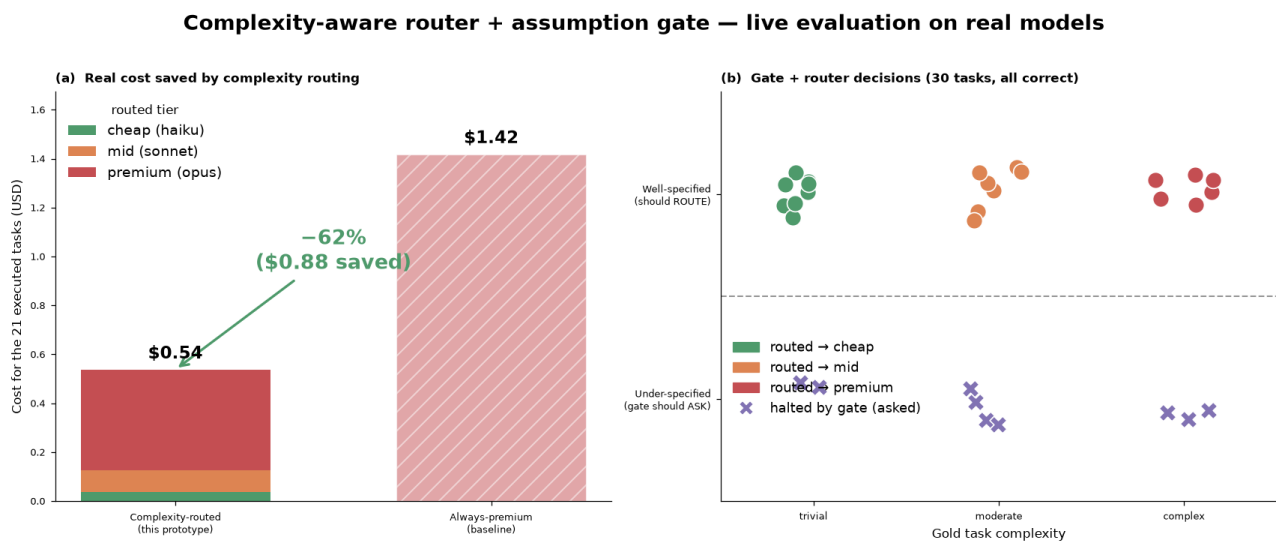


Figure 7. Live evaluation. **(a)** Real cost for the 21 executed tasks: complexity-routing spends **62% less** than always-premium on the same measured tokens, with most tasks served by the cheap and mid tiers. **(b)** All 30 decisions: every well-specified task routed to its gold tier (green/orange/red by tier), every under-specified task halted by the gate (purple x). No errors on either axis.

30/30

gate accuracy
($P = R = 1.00$)

21/21

routing exact-tier
(well-specified)

62.1%

real cost saved
vs always-premium

3/3

routed-down outputs
passed execution
tests

The gate halted all nine under-specified tasks (*“Fix the bug.”*, *“Optimize it.”*) with concrete clarifying questions and spent zero generation tokens on them; it passed all 21 well-specified tasks through. The router placed every one of those 21 on its gold tier. Across the executed workload the loop spent **\$0.54** versus **\$1.42** for always-premium — a genuine **62.1 %** reduction on real tokens.

9.3 Reading this result honestly

What this proves, and what it does not. Routing *down* only counts as a win if the cheap tier is still *correct*. So for three tasks we let the pipeline's verifier **actually execute** the generated code against real test cases: `is_prime` and `factorial` passed on the cheap tier, `merge_sorted` passed on the mid tier — no escalation needed, the saving was real and the output worked. That is the honest core of the demonstration.

What it is **not**: a benchmark. The 30-task set is small, hand-labeled, and the rubric thresholds were *tuned against it* — so the perfect separation shows the rubric *can* distinguish these cases, not that it will generalize to an arbitrary task distribution. The “retired SWE-bench Verified” finding [C6 confirmed] is a standing warning we apply to ourselves: a clean number on a self-built set is a proof of *mechanism*, not of field accuracy. A production version would calibrate on a large held-out distribution and likely *learn* the rubric weights rather than hand-set them. We report the demonstration for what it is: evidence that the two mechanisms are *buildable* and that they *behave as designed* on cases you can inspect one by one.

10 The build-opportunity map: what to build, what to skip

Eleven capabilities — five faculties, six mechanisms — and a real 2026 ecosystem to measure them against. The point of cross-referencing them is not to claim all eleven as open territory; it is the opposite. A design study earns trust by saying *what not to build* as clearly as what to build. We graded each capability against the actual Claude-Code stack — skills, hooks, plugins, MCP servers, memory backends, subagent and Agent-Teams orchestration, model-tiering gateways — as **solved**, **partial**, or **residual gap**. The full item-by-item table is the **Ecosystem appendix**; here is the decision it implies.

Already solved — do not rebuild. Two of the six mechanisms are covered well by existing tooling. **Complexity-aware routing (M1)** is handled by model tiering and gateways (LiteLLM, OpenRouter, per-agent `model:`); the only residue is transparency, not routing. **Task decomposition (M3)** is handled by subagents, Agent-Teams, and worktree fan-out. Building a new router or a new subagent framework would be reinventing shipped infrastructure — exactly the waste this project set out to avoid.

10.1 The whitespace, ranked

Ranking by *(evidence strength for the pain) × (how unsolved it is) × (buildability)*, the genuine opportunities are:

Rank	What to build	Why (grounded)	Status
1	Assumption / uncertainty gate (M2)	The project's named root failure and the field's named gap — “ <i>models rarely signal uncertainty</i> ”, the METR miscalibration result [C1] is its clearest evidence. Nothing in the stack supplies calibrated known-unknowns.	residual gap
2	Validity-anchored memory (Memory faculty)	Backends store notes (Mem0, claude-mem, Auto-Memory) but none tracks whether a stored fact was later <i>invalidated by a correction</i> ; Auto-Memory's own “memory cliff” is documented.	partial

		The write-back-only-confirmed-lessons idea is unbuilt.	
3	Mandatory pre-action impact gate (Impact-awareness)	<i>“One change to a shared utility can break dozens of packages with no cross-package awareness.”</i> Indexers retrieve; none is a <i>deterministic</i> , <i>hook-enforced</i> blast-radius check before every edit. This paper's first prototype builds exactly this.	partial
4	Outcome-validated learning loop (Learning faculty)	No tool closes the loop from a task's <i>actual outcome</i> (did the fix hold, did the PR revert) back into changed future behavior. Frozen weights guarantee it must be external.	residual gap
5	Root-cause correction gate (Self-correction)	Hooks retry; the field documents the <i>“doom loop”</i> — thrash that patches symptoms and can delete its own work while declaring success. Detecting a repeating failure signature and escalating with a diagnosis is unsolved.	partial
6	Scope-minimality check (M5)	Over-engineering is untracked outside UI; measuring footprint against the task's minimal sufficient footprint is genuine whitespace.	residual gap
7	Inline verification checkpoint (M6) · goal-drift check (M4)	Passive streaming and static anchors exist; a forced interpretive checkpoint during generation, and a continuous goal-vs-output re-check, do not.	partial

What this paper actually builds, against that map. Two prototypes, targeting the top of the list where a single session can produce real evidence. The **impact oracle** (§8) builds opportunity #3. The **router + assumption gate** (§6) builds opportunity #1 — and, honestly, the already-solved M1, whose only residue is the transparency layer we add. We prototype these two because they are the ones where *ground truth is checkable now*: an edit's blast radius against a mutation oracle, a routing decision against a hand-labeled tier, an under-specified request against a “should-ask” label. The remaining opportunities — memory validity, outcome learning, doom-loop diagnosis — are the harder research gaps, and we mark them as such rather than gesturing at them with a demo.

11 Genuinely new vs. reinvented

Intellectual honesty demands separating what this proposal *invents* from what it *assembles*. Almost every component exists somewhere; the contribution is the composition, the anchoring, and the target.

Element	Status	Assessment
External memory for a frozen model	Reinvented	RAG, MemGPT, Generative Agents ^{5,6,7} already attach stores to frozen models. We add nothing to the retrieval mechanism itself.
Fast/slow (non-parametric + LoRA) learning split	Reinvented framing	The CLS fast/slow split ^{8,9} and LoRA ¹³ are established; combining them as an agent's two learning channels is a synthesis, not a new mechanism.
Structural code graph & impact analysis	Reinvented (decades old)	PDG, slicing, CPG ^{30,28,31} are mature. Our parser is a modern, persistent re-implementation, not new theory.
Validity-anchored memory (the <i>val</i> term, Eq. 3)	Novel emphasis	Scoring and pruning memory by whether its past prediction was <i>confirmed by an external oracle</i> (test/commit), not by the model's own judgment, is the design's sharpest departure from RAG/MemGPT's self-managed stores.
Wiring exact impact analysis into the agent's pre-action gate	Novel composition	The bridge — live graph → “what depends on X” → mandatory gate before the edit — is the integration §3 showed nobody has shipped, and the one we prototyped.
Epistemic-obligation anchoring (the lens)	Novel framing	Deriving <i>which</i> safeguards are mandatory (verify-before-acting, least privilege) from a coherent epistemology, rather than bolting them on ad hoc, is a contribution of stance and organization.

In one line: **the components are largely borrowed; the loop shape, the validity anchoring, and the coding-agent target are the contribution.** That is a defensible and useful kind of novelty — it is what turns five scattered literatures into one buildable architecture.

12 Limitations & threats to validity

- **One faculty, not five.** Only impact-awareness is prototyped. Memory, learning, and imagination for code remain specified-but-unbuilt, and §3 is explicit that three of them are open research problems, not weekend engineering.
- **Static analysis has known blind spots.** The `ast`-based world-model is sound for static Python structure but conservative-to-wrong on dynamic dispatch, monkey-patching, reflection, `eval`, and dependency injection. It is single-language. Precision (0.63) reflects partly that it over-approximates; recall is its strength precisely because it errs toward inclusion.
- **Small, self-built evaluation.** Five mutations on a 10-module package plus two `stdlib` scale checks is a demonstration, not a benchmark. The mutation-derived ground truth is only as complete as the test suite (an untested consequence is invisible to it) — so true recall could be lower than measured. A convincing evaluation needs many real repositories with rich test suites and real historical edits.
- **The hard faculties may not yield to this architecture.** The substrate *assumes* that anchoring memory/learning to external oracles is sufficient; for domains or tasks without a cheap oracle, the whole premise weakens. Code is favorable; much agent work is not.
- **The lens is framing, not proof.** As stated throughout, no verse validates an engineering claim. A reader who rejects the lens loses the organizing vocabulary but none of the technical content, which stands on §3 and §8 alone.

13 Conclusion

The faculties a coding agent seems to lack — memory, learning, imagination, self-correction, impact-awareness — are not deficiencies of knowledge that scale will cure. They are structural consequences of what a frozen transformer *is*: a stateless map with fixed weights and a bounded window (Eq. 1, P1–P3). Because they follow from the *shape* of the loop, they cannot be prompted or tooled away; they can only be removed by re-wrapping the loop into a closed, stateful cycle over persistent stores, with the model left frozen inside it (Eq. 2, Fig. 1–2). We specified that substrate faculty by faculty, said honestly which parts are open research and which are engineering, and — for the

one faculty that is buildable today — shipped a running impact oracle that, against independent ground truth, never misses an affected file where the strategies a context-bounded agent actually uses miss up to half. The Qur'anic lens gave the work its spine of obligation: *know what exists before you act, verify what you are told, and hold what you can damage as a trust*. Those are not just good engineering defaults; here they are the architecture. The next step is to build the memory and learning layers against the same discipline — anchored to what can be verified, not to what the model says of itself — and to evaluate the whole loop on real repositories with real histories.

References

1. **Weston, Chopra, Bordes** (2014). Memory Networks. [arXiv:1410.3916](#). Proposes an explicit, addressable long-term memory component read/written by a neural controller, separate from model parameters.
2. **Sukhbaatar, Szlam, Weston, Fergus** (2015). End-To-End Memory Networks. [arXiv:1503.08895](#). Makes memory-network read/write fully differentiable and trainable end-to-end with weaker supervision than the original Memory Networks.
3. **Graves, Wayne, Danihelka** (2014). Neural Turing Machines. [arXiv:1410.5401](#). Couples a neural controller to an external memory matrix via differentiable attention, enabling learned algorithms like copy and sort.
4. **Graves et al.** (2016). Hybrid computing using a neural network with dynamic external memory. [Nature](#). The Differentiable Neural Computer adds dynamic memory allocation and temporal link tracking, letting a network solve graph traversal and reasoning tasks that require variable-sized structured memory.
5. **Lewis et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. [arXiv:2005.11401](#) / [NeurIPS](#). Combines a parametric seq2seq model with a non-parametric retriever over an external corpus so generation is grounded in retrieved documents rather than frozen weights alone.
6. **Packer et al.** (2023). MemGPT: Towards LLMs as Operating Systems. [arXiv:2310.08560](#). Treats the LLM context window like virtual memory, using an OS-style paging scheme (self-directed function calls) to move information between a bounded context and external storage to give the illusion of unbounded memory.
7. **Park et al.** (2023). Generative Agents: Interactive Simulacra of Human Behavior. [arXiv:2304.03442](#). Introduces a memory stream of observations plus a reflection mechanism that periodically synthesizes higher-level memories, retrieved and used to condition believable agent behavior.
8. **McClelland, McNaughton, O'Reilly** (1995). Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*. Argues the brain needs two learning systems —

a fast hippocampal one for episodic recall and a slow neocortical one for structured, interleaved consolidation — to avoid catastrophic interference.

9. **Kumaran, Hassabis, McClelland** (2016). What Learning Systems do Intelligent Agents Need? Complementary Learning Systems Theory Updated. [Trends in Cognitive Sciences, 10.1016/j.tics.2016.05.004](#). Updates complementary learning systems theory in light of deep RL and episodic-memory findings, motivating fast/slow-learning-system architectures as a computational analogue of hippocampal-neocortical interaction.
10. **Kirkpatrick et al.** (2017). Overcoming catastrophic forgetting in neural networks. [PNAS / arXiv:1612.00796](#). Elastic Weight Consolidation penalizes changes to parameters important for earlier tasks (via a Fisher-information-weighted quadratic penalty) to mitigate catastrophic forgetting during sequential training.
11. **Sun et al.** (2020). Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. [arXiv:1909.13231 / ICML 2020](#). Updates model parameters at test time via a self-supervised auxiliary task on each new input, adapting to distribution shift without labeled data.
12. **Ba, Hinton, Mnih, Leibo, Ionescu** (2016). Using Fast Weights to Attend to the Recent Past. [arXiv:1610.06258](#). Introduces a fast-changing weight matrix, updated by a Hebbian-like rule from recent activity, that lets a network attend to its own recent history at a timescale between slow weights and short-term activity.
13. **Hu et al.** (2021). LoRA: Low-Rank Adaptation of Large Language Models. [arXiv:2106.09685](#). Freezes pretrained weights and injects trainable low-rank update matrices per layer, making targeted, cheap fine-tuning/adaptation of large models feasible without touching the full parameter set.
14. **von Oswald et al.** (2022). Transformers Learn In-Context by Gradient Descent. [arXiv:2212.07677](#). Shows constructively that a transformer's forward pass over a prompt can implement steps of gradient descent on an implicit in-context loss, giving a mechanistic account of in-context learning.
15. **Dai et al.** (2022). Why Can GPT Learn In-Context? Language Models Implicitly Perform Gradient Descent as Meta-Optimizers. [arXiv:2212.10559](#). Draws a formal duality between attention and gradient-descent-based finetuning, framing in-context learning as an implicit, weight-unchanging optimization process.
16. **Cobbe et al.** (2021). Training Verifiers to Solve Math Word Problems. [arXiv:2110.14168](#). Trains a separate verifier model to score candidate solutions and select among generator samples, offloading correctness-checking to a distinct learned model rather than the generator itself.
17. **Lightman et al.** (2023). Let's Verify Step by Step. [arXiv:2305.20050](#). Shows a process-level reward model, trained on human step-by-step correctness labels, outperforms outcome-level supervision for catching reasoning errors.
18. **Shinn et al.** (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. [arXiv:2303.11366](#). Has an agent verbally reflect on task feedback/failures and stores that reflection in an episodic memory buffer used to condition subsequent attempts, substituting for gradient-based RL updates.
19. **Madaan et al.** (2023). Self-Refine: Iterative Refinement with Self-Feedback. [arXiv:2303.17651](#). Uses the same frozen LLM to generate output, critique it, and refine it iteratively in a feedback loop, with no parameter updates or external training signal.
20. **Wang et al.** (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. [arXiv:2203.11171](#). Samples multiple diverse reasoning chains and marginalizes over them by majority vote on the final answer, improving reasoning accuracy without any self-correction step per se.

21. **Huang et al.** (2023). Large Language Models Cannot Self-Correct Reasoning Yet. [arXiv:2310.01798](#). Finds that without external, ground-truth feedback, LLMs prompted to self-critique and revise their own reasoning frequently make correct answers worse, undercutting claims of intrinsic self-correction.
 22. **Ha, Schmidhuber** (2018). World Models. [arXiv:1803.10122](#). Trains a compressed generative model of an environment (VAE + RNN) and evolves a small controller entirely inside that learned latent 'dream' simulation before transferring to the real environment.
 23. **Schrittwieser et al.** (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. [arXiv:1911.08265](#) / [Nature](#). MuZero learns a model of only the quantities relevant to planning (reward, value, policy) without reconstructing observations, and plans with MCTS purely in that learned latent model.
 24. **Hafner et al.** (2023). Mastering Diverse Domains through World Models. [arXiv:2301.04104](#). DreamerV3 learns a world model from experience and trains a policy purely from imagined rollouts inside it, achieving strong performance across many domains with one fixed set of hyperparameters.
 25. **Hafner et al.** (2019). Dream to Control: Learning Behaviors by Latent Imagination. [arXiv:1912.01603](#). Backpropagates value gradients directly through imagined multi-step latent trajectories of a learned world model to learn behaviors efficiently, without interacting with the real environment.
 26. **Friston** (2010). The free-energy principle: a unified brain theory?. [Nature Reviews Neuroscience](#). Proposes that brains (and action) minimize variational free energy, unifying perception, learning, and action under a single generative-model / predictive-coding framework — active inference.
 27. **Allamanis, Brockschmidt, Khademi** (2018). Learning to Represent Programs with Graphs. [arXiv:1711.00740](#) / [ICLR 2018](#). Represents source code as a graph capturing syntax, control flow, and data flow, and applies gated graph neural networks to tasks like variable-misuse detection, giving a structural rather than purely textual code representation.
 28. **Horwitz, Reps, Binkley** (1990). Interprocedural Slicing Using Dependence Graphs. [ACM TOPLAS](#). Extends program dependence graphs across procedure boundaries to compute interprocedural slices, the structural basis for determining what code is affected by / affects a given statement.
 29. **Tip** (1994). A Survey of Program Slicing Techniques. [Journal of Programming Languages](#). Surveys the family of static and dynamic program-slicing techniques used to identify the subset of a program that can influence (or is influenced by) a given point — a core building block of change-impact analysis.
 30. **Ferrante, Ottenstein, Warren** (1987). The Program Dependence Graph and Its Use in Optimization. [ACM TOPLAS](#). Introduces the program dependence graph combining control and data dependence in one structure, the ancestor representation behind most modern static change-impact and slicing tools.
 31. **Yamaguchi, Golde, Arp, Rieck** (2014). Modeling and Discovering Vulnerabilities with Code Property Graphs. [IEEE S&P 2014](#). Introduces the code property graph, merging AST, control-flow graph, and program-dependence graph into one joint representation, queryable to discover vulnerability patterns across a codebase.
 32. **Alon, Zilberstein, Levy, Yahav** (2019). code2vec: Learning Distributed Representations of Code. [POPL 2019](#) / [PACMPL](#). Represents a code snippet as a bag of AST paths and learns a distributed embedding that predicts semantic properties such as method names, giving a learned structural fingerprint of code.
-

Appendix Evidence map — the twelve statistics, re-grounded

Each load-bearing pain-point statistic in the 2026 discourse, traced to its primary source and graded. **Confirmed** = independent primary source; **vendor-reported** = real but sourced only to a party with a commercial stake; **unverifiable** = could not corroborate in primary form. Full source records and paper-use recommendations are in `evidence_map.json`.

ID	Claim	Primary source	Supports	Status
C1	Experienced open-source developers were 19% SLOWER with AI while believing they were ~20% faster (also forecast 24% speedup beforehand).	Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity	M2 (assumption/ uncertainty - miscalibration), P3/ self-correc	confirmed
C2	Trust in AI accuracy fell from 40% to 29% (or 46% actively distrust vs 33% trust per detailed breakdown); favorability 72%->60%; 66% say AI answers are 'almost right, but not quite	2025 Stack Overflow Developer Survey (49,000+ respondents, 177 countries)	M2 uncertainty/ calibration; M6 inline verification; overall	confirmed
C3	45% of AI-generated code samples introduce an OWASP Top-10 vulnerability; a widely-repeated '2.74x more vulnerabilities than human-written code' figure.	2025 GenAI Code Security Report	M5 anti-over-engineering / code-quality thesis; security-ver	vendor-reported
C4	~8x rise (2024 vs prior years) in frequency of duplicated/copy-pasted code blocks; copy-paste overtaking refactored ('moved') code for the first time.	AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones / AI Copilot Code Qua	M5 anti-over-engineering; long-term codebase impact-awarenes	vendor-reported

C5	AI's primary role is as an 'amplifier' - magnifying high performers' strengths and low performers' dysfunctions; AI continues to increase delivery instability even as adoption beco	State of AI-assisted Software Development 2025 (DORA Report)	Systemic framing for all six mechanisms (org context matters)	confirmed
C6	OpenAI retired SWE-bench Verified (Feb 2026) after finding at least 59.4% of audited (hard/failed) problems had flawed test cases and/or training-data contamination; large score ga	Why SWE-bench Verified no longer measures frontier coding capabilities	M6 inline verification / benchmark-trust thesis; supports 'A	confirmed
C7	Median time in PR review up 441.5%; incidents-per-PR up 242.7%; bugs per developer up 54%; 31.3% more PRs merged with no review at all.	The AI Engineering Report 2026: The Acceleration Whiplash	M6 inline verification; review-bottleneck / verification-lay	vendor-reported
C8	Developers who delegate code generation to AI score 17% lower on comprehension tests, based on 'Anthropic's own research (~400,000 Claude Code sessions)'. TWO DIFFERENT STUDIES ARE BEING CONFLATED: (a) 'How AI Impacts Skill Formation' by Judy Ha		M6 inline verification / skill-atrophy thesis (comprehension)	unverifiable
C9	96% of developers don't fully trust AI-generated code is functionally correct, yet only 48% always verify it before committing (a 48-point 'verification gap'/'verification debt').	State of Code Developer Survey report 2026	M6 inline verification - the central named gap the mechanism	vendor-reported
C10	JetBrains' 2025 survey found 77% of developers still manually correct AI	The State of Developer Ecosystem 2025	M4 goal-anchoring / M5 anti-over-	unverifiable

	output for project conventions every session.		engineering (convention dri	
C11	A standard MCP setup (few servers) can consume ~72% of a 200K-token context window before work begins; tool-selection accuracy drops from ~43% to below ~14% as tool count scales ('	(a) 72%-context-window claim: no formal paper found, only recurring blog anecdotes (Scott	M1 complexity-aware routing / M3 task decomposition (context	vendor-reported
C12	LLM evaluators recognize and favor their own generations - self-preference bias correlates with self-recognition ability - motivating why 'AI verifying AI' is structurally weak.	LLM Evaluators Recognize and Favor Their Own Generations	M6 inline verification (why an LLM cannot be its own sole ve	confirmed

Appendix Ecosystem map — faculties & mechanisms vs. the real stack

Each faculty and mechanism graded against the actual 2026 Claude-Code stack (skills, hooks, plugins, MCP servers, memory backends, subagent/Agent-Teams orchestration, model-tiering gateways). **Solved** = covered well by existing tooling; **partial** = tooling exists but leaves a named gap; **residual-gap** = genuine whitespace. Full tool lists and citations are in `ecosystem_map.json`.

Capability	Existing tooling (sample)	Status	Residual gap	What we'd add
Memory (across sessions) faculty	CLAUDE.md; Auto Memory; Mem0; claude-mem; Hindsight	partial	Pain-points report names this directly: 'no standard, tool-agnostic, durable memory layer that reliably persists project knowledge,	<i>Validity-anchored memory: stored facts carry a confirmed/ discredited state updated by verified outcomes, not a static note-dump --</i>

			decisions, and corrections across sessions, tools, and teammates' (Open gap #2), with JetBrains finding 77%	<i>addresses the correction time</i>
Learning (from outcomes) faculty	Auto Memory 'feedback' memory type; Superpowers subagent-driven-development two-stage review; ccusage / usage trackers	residual-gap	No tool in the stack closes the loop from a task's actual outcome (did the fix hold, did the test stay green, was the PR reverted) back into changed future behavior. Auto Memory's feedback type is note storage, not an evaluated lesson; froz	<i>Outcome-validated learning loop: capture a task's actual downstream result, verify it independently, and write back only confirmed lessons (not raw logs) into t</i>
Imagination (simulate consequences before acting) faculty	Plan mode / built-in Plan subagent; Explore subagent; Superpowers brainstorm -> plan -> TDD -> subagent-dev -> review	partial	These are textual/ symbolic planning steps within the stack itself, not a simulation of an edit's actual downstream effects -- no hook, skill, or MCP server in the stack landscape traces a dependency graph before a change is made. (The pain-	<i>A pre-action impact simulation step -- dependency-graph/ call-graph traversal that predicts affected files/tests before code is written, feeding predicted blast</i>
Self-correction faculty	Stop hooks; PreToolUse blocking hooks; Playwright's healer agent; TDD RED-GREEN-REFACTOR; / security-review and pr-review-toolkit	partial	These are retry/gate mechanisms, not diagnosis. The pain-points report documents the 'doom loop' by name -- an agent that 'makes a mistake, tries to fix it, makes it worse' and can even delete its own changes while declaring success -- and	<i>A root-cause-aware correction gate that distinguishes genuine progress from thrash (same failure signature repeating), halts and escalates with a diagnosis inst</i>

Impact-awareness (what exists in the codebase / what an edit affects) faculty	filesystem and memory; Context7; sequential-thinking MCP server; sandboxing's 'trust verification for new codebases/MCP servers'	partial	These stack primitives expose retrieval (docs, a memory graph, filesystem access) that the model may or may not consult -- none is a deterministic, mandatory gate run before every edit, and none computes blast radius. (The pain-points repor	<i>A MANDATORY pre-action impact gate -- a hook-enforced (not LLM-judged) dependency-graph query that runs before every edit is applied, not an optional retrieval</i>
M1 Complexity-aware routing mechanism	Model tiering; Per-agent model: field in .claude/agents/; LLM gateways: LiteLLM, Portkey, OpenRouter; Named orchestration pattern: 'orchestrator-classifies-then-routes to Haiku/Sonnet/Opus by complexity'	solved	The routing decision is set at config time (a developer picks model: haiku for an agent) or by the gateway's own cost logic -- it is not a transparent, per-task, auditable classification the user can see and override before dispatch.	<i>A transparent complexity classification surfaced to the user before dispatch, making the routing decision auditable rather than a silent config default.</i>
M2 Assumption / uncertainty mechanism	Reasoning models acting as an internal review pass; Context7; Spec-driven development	residual-gap	Named directly and unsolved in the pain-points report: 'Models rarely signal uncertainty or say I can't do this' -- the report's own build opportunity is 'calibrated-confidence and known-unknowns tooling -- agents that flag low-confidence r	<i>This is the paper's named root failure: an assumption/uncertainty gate that requires the model to enumerate its unstated assumptions and ask before proceeding w</i>
M3 Task / session decomposition mechanism	Subagents; Agent Teams; git worktrees for parallel-agent isolation; Parallel fan-out	solved	Mature, well-tooled pattern. The remaining gap is small: deciding the decomposition boundary itself (what	<i>Automatic decomposition-boundary detection -- deciding when to fork a subagent/session vs. keep work in one</i>

	orchestration pattern; Superpowers subagent-driven-development		counts as independent vs. needs shared context) is still a manual/heuristic judgment call by the developer, not something any listed t	<i>context, rather than leaving that call to developer</i>
M4 Goal-anchoring mechanism	CLAUDE.md; Spec-driven development; /goal command; Task lists in Agent Teams	partial	These anchors are loaded once and are static. The report documents the anchor decaying over a session: 'circular reasoning at 20% [context usage], context compression wiping scrollbar at 40%,' and Open gap #1 states plainly that 'specs dri	<i>A continuous goal-drift check that periodically re-validates in-progress output against the original stated objective, rather than loading the goal once and tru</i>
M5 Anti-over-engineering mechanism	frontend-design skill's minimalism discipline; TDD RED-GREEN-REFACTOR; pr-review-toolkit / CodeRabbit / Greptile review agents	residual-gap	The frontend-design skill's discipline is explicitly UI-only; no general-purpose backend/architecture tool measures unnecessary abstraction, premature generalization, or scope creep against the stated task. GitClear's tracked defects (8x du	<i>A scope-minimality check that compares an implementation's footprint (files touched, abstractions introduced) against the stated task requirement and flags addi</i>
M6 Inline verification mechanism	Real-time thinking/ tool-call streaming; Self-QA pattern; PostToolUse hooks; Plan mode human-in-the-loop gating	partial	Streaming and immediate post-edit hooks give passive visibility but do not require a human interpretive checkpoint DURING generation. The pain-points report's central statistic is the '48-point gap': 'Sonar's	<i>A mandatory inline checkpoint that surfaces a human-checkable claim or diff at each meaningful generation step, shifting verification earlier instead of batchin</i>

			2026 report: 96% of developers	
--	--	--	--------------------------------	--

Appendix Qur'anic-lens mapping table

The complete 14-row mapping (12 load-bearing, 2 metaphor). Canonical Arabic and translations retrieved from quran.ai; full text, tafsir references, and design principles in the companion artifact `quran_lens.json`. Caveat: this table is a design lens, not technical or theological authority — see §5.

Concept / verse	Retrieved gloss	Faculty	Design principle (abbrev.)	Type
17:36 — lā taqfu (do not pursue without knowledge)	Do not follow blindly what you do not know to be true: ears, eyes, and heart, you will be questioned about all these.	IMPACT-AWARENESS	Before any code mutation (file write, delete, refactor), the agent must run a pre-action verification gate that checks: (1) what entities in the codebase wil...	load-bearing
49:6 — tabayyun (verify reports before acting)	Believers, if a troublemaker brings you news, check it first, in case you wrong others unwittingly and later regret what you have done,	SELF-CORRECTION	The agent architecture must include a verification gate between receiving information (from context, tool output, or its own prior reasoning) and acting on it	load-bearing
2:31-32 — taʿlīm al-asmāʾ (He	(2:31) He taught Adam all the names	WORLD-MODEL	The agent must maintain a	load-bearing

Concept / verse	Retrieved gloss	Faculty	Design principle (abbrev.)	Type
taught Adam the names of all things)	[of things], then He showed them to the angels and said, ‘Tell me the names of these if you truly [think you can].’ (2:32) They sai...	(codebase graph)	structured representation of what exists in the codebase — a graph of files, functions, classes, dependencies, and their relationsh...	
20:114 — rabbi zidnī ‘ilmā (My Lord, increase me in knowledge)	exalted be God, the one who is truly in control. [Prophet], do not rush to recite before the revelation is fully complete but say, ‘Lord, increase me in knowledge!’	CONTINUAL LEARNING	The agent's knowledge must be treated as perpetually incomplete, with an explicit mechanism for incremental growth	load-bearing
96:1-5 — iqra’ / ‘allama bi-l-qalam (Read; taught by the pen)	(96:1) Read! In the name of your Lord who created: (96:2) He created man from a clinging form. (96:3) Read! Your Lord is the Most Bountiful One (96:4) who taught by [me...	PERSISTENT MEMORY	Knowledge must be externalized to survive beyond the moment of computation	load-bearing
39:9 — hal yastawī lladhīna ya‘lamūna wa-lladhīna lā ya‘lamūn (are those who know equal to those	What about someone who worships devoutly during the night, bowing down, standing in prayer, ever mindful of the life to come, hoping for	MEMORY + LEARNING (epistemic grounding)	An agent that retains and learns from experience is categorically more capable and more trustworthy than	metaphor

Concept / verse	Retrieved gloss	Faculty	Design principle (abbrev.)	Type
who do not know?)	his Lord's mercy? Say, 'How ca...		one that does not — this is not a nice-to-have but a...	
4:82 — tadabbur al-Qur'ān (do they not reflect deeply upon the Quran; inconsistency as a sign of non-divine origin)	Will they not think about this Quran? If it had been from anyone other than God, they would have found much inconsistency in it.	SELF-CORRECTION (metacognitive controller)	The agent must run self-consistency checks on its own output before committing it	load-bearing
47:24 — tadabbur / aqfāl 'alā qulūb (do they not ponder, or are there locks on their hearts?)	Will they not contemplate the Quran? Do they have locks on their hearts?	SELF-CORRECTION (iterative refinement)	Reflection must be a deliberate, repeatable operation, not a one-pass judgment	metaphor
33:72 — al-amāna (the Trust offered to heavens, earth, mountains; borne by the human)	We offered the Trust to the heavens, the earth, and the mountains, yet they refused to undertake it and were afraid of it; mankind undertook it—they have always been ...	STEWARDSHIP ETHICS (bounded autonomy)	An agent that can modify a codebase bears a trust (amāna) — it has accepted responsibility for something it can damage	load-bearing
CONCEPT: 'ilm → fahm → ḥikma (knowledge → understanding → wisdom)	A classical epistemological hierarchy: 'ilm is raw knowledge (facts, data, observations); fahm is comprehension	MEMORY + LEARNING (data architecture)	The agent's memory/learning stack must be layered, not flat	load-bearing

Concept / verse	Retrieved gloss	Faculty	Design principle (abbrev.)	Type
	(grasping the relations between facts, seeing why); hik...			
CONCEPT: ḥifẓ + murājaʿa (preservation + spaced review/ revision)	The classical Quranic memorization discipline: ḥifẓ is initial encoding and faithful preservation of the text; murājaʿa is the regular, spaced revision that prevents d...	PERSISTENT MEMORY (consolidation & maintenance)	Memory is not write-once	load-bearing
CONCEPT: tadabbur (deep, structured reflection — returning to examine consequences)	Tadabbur is not casual thought; its root d-b-r relates to 'what is behind' or 'what follows' — examining the consequences and deeper implications of something. In Qura...	SELF-CORRECTION (metacognitive controller)	The metacognitive controller is a tadabbur loop: after the agent generates a plan or action, the controller examines what comes after (d-b-r) — what are the ...	load-bearing
CONCEPT: tabayyun (verification of reports before acting on them)	Tabayyun is the act of seeking clarity and verification before acting on received information. In 49:6, it is commanded as a mandatory step between receiving a report ...	SELF-CORRECTION (verification gate)	The verification gate is the architectural realization of tabayyun	load-bearing
CONCEPT: amāna (trust,	Amāna is the trust or responsibility that is		When an agent is granted	load-bearing

Concept / verse	Retrieved gloss	Faculty	Design principle (abbrev.)	Type
stewardship, accepted responsibility)	accepted voluntarily and carries accountability. In 33:72, it is described as something so weighty that the heavens, earth...	STEWARDSHIP ETHICS (safe operation)	access to a codebase, it accepts an amāna — a trust that carries accountability proportional to its capability	

Companion artifacts. This paper is accompanied by: the runnable `impact_oracle_src.zip` (world-model + oracle + demo + 36-test suite); the impact-graph and evaluation figures; the structured literature `gap_map.json` and 32-entry `references.json`; and the `quran_lens.json` mapping table. All results are reproducible via the included `demo.py`.

On the lens. Qur'anic text throughout was retrieved from the quran.ai service and presented directly, not paraphrased. It is used as an epistemological and ethical design lens only; every technical claim rests on the cited literature and the prototype evaluation, independent of the lens.